UNIVERSIDAD NACIONAL DEL SUR
Departamento de Ciencias e Ingeniería de la Computación
Compiladores e Intérpretes — Segundo Cuatrimestre de 2026

Compilador de MiniJava

Etapa 2 — Analizador Sintáctico

Descendente Recursivo Predictivo

Agustín Paul
L.U. 147524

Informe de entrega

Transformación de la gramática a LL(1), paso a paso.
Las nueve extensiones por logro y su verificación.

17 de septiembre de 2026

Índice

1. Alcance **1**
- 1.1. Qué está y qué no está en este informe . . . 1
- 1.2. Qué chequea y qué no chequea un analizador sintáctico . . . 1

2. Logros perseguidos **2**

3. Notación **2**
- 3.1. El criterio LL(1) . . . 3
- 3.2. Dos defectos distintos que conviene no confundir . . . 3

4. Diagnóstico **4**
- 4.1. Cuántos de esos dieciséis son ambigüedades . . . 4
- 4.2. Las dos ambigüedades . . . 5
- 4.3. Una tercera ambigüedad heredada, y una que agrega un logro . . . 5

5. Transformaciones **6**
- 5.1. Orden de trabajo y criterios . . . 7
- 5.2. Eliminación de recursión a izquierda . . . 7
  - 5.2.1. R1 — `<ListaArgsFormales>` . . . 7
  - 5.2.2. R2 — `<ExpresionCompuesta>` . . . 8
  - 5.2.3. R3 — `<Referencia>`, y el conflicto que la transformación genera . 8
- 5.3. Factorización a izquierda . . . 9
  - 5.3.1. F1 — `<Expresion>` . . . 9
  - 5.3.2. F2 — `<Sentencia>`: resolver la ambigüedad . . . 9
  - 5.3.3. F3 — `<If>`: factorizar, y después resolver el *dangling else* . . . 10
  - 5.3.4. F4 — `<DimensionesConTamanio>` . . . 10
  - 5.3.5. F5 — `<ListaExps>` . . . 10
  - 5.3.6. F6 — `<Miembro>`: factorización profunda . . . 11
  - 5.3.7. F7 — `<Primario>`: el prefijo **idMetVar** . . . 11
  - 5.3.8. F8 — `<Primario>`: el prefijo **new**, a dos niveles . . . 12
- 5.4. Estado del núcleo . . . 12

6. Extensiones por logro **12**
- 6.1. Visibilidad Mejorada E2 . . . 13
- 6.2. Atributos Inicializados y Variables locales Clásicas E2 . . . 14
- 6.3. Fors! E2 . . . 15
- 6.4. Operador Ternario E2 . . . 16
- 6.5. Operadores Posifijos E2 . . . 16
- 6.6. MiniLambdas E2 . . . 16
- 6.7. Inicializacion de Arreglos E2 . . . 20
- 6.8. Genericidad Avanzada E2 . . . 22
- 6.9. Floats (*carry-over* de la Etapa 1) . . . 22
- 6.10. Impacto sobre el analizador léxico . . . 23

7. Gramática final **23**

8. Ambigüedades resueltas **31**

9. El analizador sintáctico **31**
9.1. La técnica 31
9.2. Las clases 32
9.3. Los tipos de error que detecta 33
9.4. El formato del reporte 34

10. Cómo compilar y correr **34**

11. Verificación **35**
11.1. Condición LL(1) 35
11.2. Ambigüedad frente a conflicto de predicción 35
11.3. Registro de ambigüedades 35
11.4. Casos de aceptación y rechazo 36
11.5. Validación contra los casos de la cátedra 37
11.6. Tres reconocedores independientes de la misma gramática 37
11.7. Equivalencia de lenguajes con la gramática de la cátedra 38
11.8. Contraste con un corpus ajeno 38
11.9. El analizador cita exactamente la gramática 40

12. Decisiones tomadas con criterio propio **40**
12.1. Dónde la especificación admitía más de una lectura 40
12.2. Qué verifica el analizador y qué queda para el semántico 42
12.3. Decisiones de implementación 44

13. Herramientas desarrolladas **45**

1 Alcance

Este informe documenta la Etapa 2 del compilador de MiniJava: el **Analizador Sintáctico Descendente Recursivo Predictivo**. Su objeto central es la **gramática**: se parte de la que publica la cátedra, se la diagnostica, se la corrige a LL(1) paso a paso, se la extiende con los nueve logros perseguidos y se verifica el resultado.

El criterio de exposición es que **cada corrección quede localizada en un paso concreto**. Por eso las transformaciones no se presentan como un salto de la gramática vieja a la nueva, sino en cuatro pasos:

1. la **regla original**, tal cual el documento de la cátedra;
2. el **defecto** detectado, con los conjuntos de Primeros que se solapan;
3. la **transformación** aplicada, citando el esquema de la teoría que la justifica;
4. el **resultado** y la verificación de que los Primeros quedan disjuntos.

1.1 Qué está y qué no está en este informe

Está la gramática y su tratamiento, y está el analizador que sale de ella: cómo compilarlo y correrlo (sección 10), cómo está construido y qué errores detecta (sección 9), qué logros persigue (sección 2) y qué decisiones hubo que tomar donde la especificación admitía más de una lectura (sección 12).

1.2 Qué chequea y qué no chequea un analizador sintáctico

Conviene fijarlo desde el principio, porque delimita todo lo que sigue.

El analizador sintáctico verifica que la secuencia de tokens esté en un orden que la gramática pueda generar: la estructura de clases, interfaces, miembros, bloques y sentencias; que los delimitadores abran y cierren; que una expresión esté bien formada; que un método tenga cuerpo y un método de interfaz no.

No verifica nada que dependa del *significado* de los nombres. Todo lo siguiente atraviesa el análisis sintáctico sin objeción y corresponde al analizador semántico:

Pasa el análisis sintáctico	**Por qué es semántico**
`int x = "hola";`	compatibilidad de tipos
`noDeclarada = 5;`	declaración y alcance de nombres
`5 = x;`	que el lado izquierdo sea asignable
`class A extends UnaInterfaz {}`	que el padre sea realmente una clase
`m(1,2,3)` con `m` de un argumento	aridad contra la declaración
`int x; int x;`	declaraciones duplicadas
`return;` en un método `int`	compatibilidad del valor de retorno
una lambda que usa una variable de afuera	captura de variables

El último caso merece atención porque afecta a un logro. El enunciado de *MiniLambdas E2* prohíbe la captura de variables locales y de parámetros, pero eso **no es decidible**

sintácticamente: el analizador no sabe qué nombres están en alcance en cada punto. Lo único de ese logro que la gramática puede imponer es la *forma* — parámetros sin tipo, cuerpo de una única expresión, sin llaves — y así se hace. La restricción sobre la captura queda anotada como responsabilidad del analizador semántico.

2 Logros perseguidos

Se persiguen **todos los logros de la etapa**: los diez que son implementables por código —más el *carry-over* de *Floats* de la Etapa 1— y los tres que no lo son.

Los diez están **siempre activos**. El compilador reconoce MiniJava con todos los logros de la Etapa 1 y de la Etapa 2 incorporados, y no es configurable: no hay opción de línea de comandos ni propiedad de sistema para elegir con cuáles correr, y la gramática no tiene producciones que aparezcan y desaparezcan. Hubo una versión anterior en la que cada logro era un interruptor independiente, verificada sobre las 2^{10} configuraciones; se revirtió por indicación del profesor, porque complejizaba la gramática sin necesidad y cada etapa nueva obligaría a rehacer el trabajo con más chance de error.

Logro	**Dónde está**
MiniLambdas E2	§6: el bloque de `<ParentizadaOLambda>`, la lambda en el nivel de `<Expresion>`, y la frontera con el analizador semántico. Requiere el token `->`.
Variables locales Clásicas E2	La factorización de **idClase** en `<Sentencia>`.
Visibilidad Mejorada E2	`<Visibilidad>` al frente del miembro, y a los dos lados del **static**. Requiere el token **private**.
Fors! E2	El bloque `<For>` entero, estándar y de iteradores. Requiere el token **for**.
Operador Ternario E2	La capa `<ExpresionCondicional>`. Requiere el token **?**, que en la Etapa 1 era un error léxico.
Operadores Posifijos E2	`<PosfijoOpcional>`. Único logro que no toca el léxico: `++` y `--` ya eran tokens de la Etapa 1.
Atributos Inicializados	`<InicializacionDeAtributo>`.
Inicializacion de Arreglos E2	`<InicializadorDeArregloOpcional>`, anidable y sólo bajo **new**, colgado de las dimensiones con tamaño.
Genericidad Avanzada E2	Genéricos anidados y la notación diamante.
Recuperación Modo Pánico	No toca la gramática: es enteramente implementación.
Floats (Etapa 1)	`<TipoPrimitivo>` → **float** y `<Primitivo>` → **floatLiteral**.

Los otros tres —*Entrega Anticipada*, *Imbatibilidad* y *Well Done!*— también se persiguen, pero no se implementan por código: no dependen del compilador.

3 Notación

Se usa la misma notación BNF-extendida del documento *Reglas de Sintaxis de MiniJava* de la cátedra, con la misma convención tipográfica, para que las reglas de este

informe se puedan comparar renglón a renglón con las de aquél.

Elemento	Significado
terminal	símbolo terminal
`<Clase>`	símbolo no terminal (primera letra mayúscula)
ϵ	la cadena vacía
`<X> ::=` α	una producción, con α una secuencia de terminales y no terminales
`<X> ::=` α | β	abreviación de `<X> ::=` α y `<X> ::=` β
Primeros(β)	conjunto de terminales que pueden aparecer primeros en β

El no terminal inicial es `<Inicial>`.

La gramática es **una sola**: describe MiniJava con todos los logros incorporados y no tiene producciones que aparezcan y desaparezcan según nada (sección 2). Cuando una sección de este informe atribuye una producción a un logro, es para decir de dónde salió, no para marcar una variante.

3.1 El criterio LL(1)

Una gramática es LL(1) si, para todo no terminal A con producciones $A \to \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$, se cumple para todo $i \neq j$:

$$\text{Primeros}(\beta_i) \cap \text{Primeros}(\beta_j) = \varnothing$$

Sólo con esa condición un único token de anticipación alcanza para elegir la producción correcta sin necesidad de *backtracking*. Cuando la condición no se cumple, la gramática debe adaptarse **sin cambiar el lenguaje que genera**, que es la restricción que gobierna todas las transformaciones de este informe.

3.2 Dos defectos distintos que conviene no confundir

Buena parte del trabajo consistió en separar dos cosas que se parecen y no son lo mismo.

- **Ambigüedad — el lenguaje está mal descripto**

  Existe una cadena con **dos árboles de derivación distintos**. No queda determinado qué significa el programa. No siempre se puede arreglar reescribiendo: a veces hay que *elegir* cuál de los dos significados vale.

- **Conflicto LL(1) — el analizador no puede predecir**

  Dos producciones del mismo no terminal comparten Primeros. La cadena sigue teniendo **un solo** árbol; lo que falla es que el analizador no puede saber cuál producción aplicar mirando un token. Se corrige factorizando o eliminando recursión, y el lenguaje generado **no cambia**.

Todo par de producciones que comparte prefijo izquierdo produce un conflicto LL(1); no todo par que comparte prefijo izquierdo es ambiguo. Si los dos lados derivan lenguajes disjuntos, no hay ninguna cadena con dos árboles.

Un criterio suficiente y barato para probar esa disyunción es comparar los **rangos de longitud derivable**: si no se solapan, los lenguajes son disjuntos y no puede haber ambigüedad. Se aplica en la sección 4 y decide varios de los casos.

4 Diagnóstico

La gramática de la cátedra **no es LL(1)**. Verificando la condición Primeros(β_i) ∩ Primeros(β_j) = ∅ sobre todas las producciones aparecen **dieciséis conflictos** repartidos en **diez no terminales**.

#	No terminal	Defecto	Intersección
1	`<Miembro>`	prefijo común profundo	**boolean**, **char**, **int**, **idClase**, **idGen**
2	`<ListaArgsFormales>`	recursión a izquierda	ídem
3	`<Sentencia>`	**ambigüedad**	todos los de `<Expresion>`
4	`<If>`	prefijo común	**if**
5	`<Expresion>`	prefijo común	todos los de `<ExpresionCompuesta>`
6	`<ExpresionCompuesta>`	recursión a izquierda	todos los de `<ExpresionBasica>`
7–12	`<Referencia>`	recursión a izquierda, tres alternativas (seis pares)	todos los de `<Primario>`
13	`<Primario>`	prefijo común	**idMetVar**
14	`<Primario>`	prefijo común a dos niveles	**new**
15	`<DimensionesConTamanio>`	prefijo común	**[**
16	`<ListaExps>`	prefijo común	todos los de `<Expresion>`

4.1 Cuántos de esos dieciséis son ambigüedades

Siguiendo el criterio de la sección 3.2, se clasificó cada par. El resultado es que **sólo dos** de los dieciséis son ambigüedades reales; los otros catorce son conflictos de predicción que se corrigen mecánicamente sin alterar el lenguaje.

Par	Longitudes derivables	Veredicto
`<VarEncadenada>` / `<MetodoEncadenado>`	2..2 vs. 4..∞	no ambiguo
`<AccesoVar>` / `<LlamadaMetodo>`	1..1 vs. 3..∞	no ambiguo
`<CrearArreglo>` / `<LlamadaConstructor>`	se solapan	no ambiguo (argumento posicional)
`<Atributo>` / `<Metodo>`	se solapan	no ambiguo (argumento posicional)
`<Asignacion>` / `<Llamada>`	se solapan	**ambiguo**
las dos producciones de `<If>`	se solapan	**ambiguo**

El caso de `<VarEncadenada>` y `<MetodoEncadenado>` es el más ilustrativo y el que más se confunde. `<VarEncadenada>` deriva **exactamente** la cadena **. idMetVar**: dos tokens, sin alternativas. `<MetodoEncadenado>` es **. idMetVar** `<ArgsActuales>`, y `<ArgsActuales>` **no es vaciable** — siempre aporta al menos **(** y **)** — de modo que deriva cuatro tokens como mínimo. Los rangos no se solapan, **ninguna cadena puede estar en los dos lenguajes** y por lo tanto ninguna cadena tiene dos árboles. Comparten prefijo izquierdo; eso rompe LL(1) y se arregla factorizando, pero no es ambigüedad.

Para los dos pares cuyos rangos sí se solapan el argumento es posicional en lugar de por longitud: `<CrearArreglo>` y `<LlamadaConstructor>` comparten **new** y el tipo, pero `<DimensionesConTamanio>` siempre arranca con **[** y `<ArgsActuales>` siempre con **(**, así que las cadenas difieren en el token inmediatamente posterior al tipo; `<Atributo>` y `<Metodo>` comparten `<Tipo>` **idMetVar** y siguen con **;** y con **(** respectivamente.

4.2 Las dos ambigüedades

Ambigüedad — `<Asignacion>` y `<Llamada>` son la misma producción

```
<Asignacion> ::= <Expresion>
<Llamada> ::= <Expresion>
```

Los cuerpos son **idénticos**, y `<Sentencia>` los usa en dos alternativas también idénticas. En consecuencia *toda* sentencia-expresión tiene dos árboles de derivación — uno por `<Asignacion>` y otro por `<Llamada>` — y la gramática no ofrece ningún criterio para elegir. Presumiblemente están así para documentar la intención: en una sentencia-expresión uno espera una asignación o una llamada. Pero sintácticamente `x = 5` y `f()` son las dos una `<Expresion>`, y la gramática no puede distinguirlas.

Ambigüedad — el *dangling else*

En `if (a) if (b) s1; else s2;` el **else** puede ligar al **if** interno o al externo: dos árboles para la misma cadena. Es la ambigüedad clásica, presente tal cual en la gramática.

Ambas se resuelven en la sección 5, y en los dos casos hay que *decidir* un significado, no sólo reescribir.

4.3 Una tercera ambigüedad heredada, y una que agrega un logro

El criterio LL(1) de la cátedra —disjunción de Primeros— es el que decide si el analizador puede predecir, y es el único que interviene en la implementación. Pero el checklist de aceptación de Clase 4, slide 41, tiene un ítem más: que la gramática **no sea ambigua, o sea arreglable**. Verificarlo requiere mirar, además, qué puede seguir a un no terminal anulable. Se hizo esa verificación, y apareció una ambigüedad más además de las dos anteriores.

Decisión — los Siguientes son un diagnóstico, no parte del método

Detectar una ambigüedad exige mirar el contexto en que aparece un no terminal anulable. Ese cálculo se hizo **una sola vez y fuera del compilador**, con la herramienta descripta en la sección 11, con el único fin de auditar el ítem del checklist.

El analizador decide sólo con Primeros, como enseña la cátedra: los Siguientes no aparecen en la gramática, ni en la elección de producciones, ni en una línea del código.

Una vez detectada, cada ambigüedad se resuelve con una regla de implementación —la vía que autoriza Clase 4, slide 49— y no vuelven a intervenir.

Tercera ambigüedad: `new int[3][4]`, heredada de la cátedra. Después de crear un arreglo, un **[** puede ser otra dimensión de la creación o un acceso al arreglo recién creado. Las dos derivaciones existen ya en la gramática original, con producciones que están todas en el documento:

Creación 2-D: `<CrearArreglo>` ⇒ **new** `<TipoBase> <DimensionesConTamanio>` ⇒ **new int** [3] [4]

Creación 1-D indexada: `<Referencia>` ⇒ `<Referencia> <AccesoArreglo>` ⇒ `<Primario>` [4] ⇒ **new int** [3] [4]

La cuarta: el cuerpo de la lambda. `x -> a = b` puede leerse `x -> (a = b)` —el cuerpo se come la asignación— o `(x -> a) = b`. Afecta a un solo no terminal, `<RestoExpresion>`, que es la única cola externa que todavía compite con la cola del cuerpo. Se resuelve voraz, que es lo que hace Java y lo que un descendente recursivo hace por construcción: el método del cuerpo llama a `<Expresion>` y ésta avanza mientras pueda.

I Decisión — dónde vive la lambda: no es un operando

La ubicación que se impone sola al leer la especificación del logro es `<Primario>`: la lambda aparece donde aparece un valor, y `<Primario>` es el no terminal de los valores. Se evaluó y se descartó, porque `<Primario>` es la ranura de un *operando* y eso trae dos consecuencias, la segunda grave:

1. la cola de la expresión que la contiene competía con la cola de su cuerpo, y la ambigüedad anterior se multiplicaba por cinco: `<RestoExpresion>` (=), `<RestoExpresionCompuesta>` (los trece binarios), `<RestoReferencia>` (. y [), `<RestoCondicional>` (?) y `<PosfijoOpcional>` (++, --);
2. **el lenguaje reconocido era estrictamente más grande que el de Java**. Si una lambda es un `<Primario>`, entonces es un operando válido, y `1 + x -> x`, `a == x -> x`, `- x -> x`, `(x -> x) + 1` y `(x -> x).m()` pasan el análisis. En Java los cinco son error —*«lambda expression not expected here»*, verificado con `javac`—, porque el JLS define *Expression: LambdaExpression | AssignmentExpression*: la lambda es una alternativa del nivel **más alto** de la jerarquía, nunca un *Primary*. Dicho de otro modo: -> no tiene precedencia contra los operadores porque no compite con ellos.

Por la regla «lo que no está especificado se comporta como en Java», el segundo punto decide: la lambda se produce en el nivel de `<Expresion>`, como en el JLS. El costo es factorizar las dos cabezas que comparte con una expresión normal —**idMetVar** y **(**— contra la cola de expresión, con la misma técnica que `<ExpresionSinIdClase>`: catorce producciones nuevas, ninguna lógica duplicada, porque todas las colas se comparten. El beneficio es doble: el lenguaje pasa a ser el de Java en las cinco formas de la tabla, y cuatro de las cinco ambigüedades **desaparecen de raíz** en vez de resolverse por regla.

5 Transformaciones

5.1 Orden de trabajo y criterios

Primero eliminar la recursión a izquierda, después factorizar. El orden importa porque eliminar la recursión genera producciones nuevas sin factorizar; la sección 5.2.3 es el caso concreto donde eso ocurre.

No factorizar de más. Las reglas originales tienen significado y el analizador semántico de la Etapa 3 lo va a necesitar. En la práctica eso se tradujo en cuatro criterios:

- se conserva el nombre original siempre que la regla sobreviva;
- los no terminales nuevos llevan **nombres semánticos en español** (`<RestoExpresionCompuesta>`, no `<ExpresionCompuesta'>`);
- cuando dos no terminales originales se fusionan a la fuerza, se dice cuáles y por qué;
- ante la disyuntiva entre fusionar y **insertar un no terminal**, se inserta.

Decisión — insertar el no terminal, no fusionar

Al factorizar un prefijo hay dos caminos. *Fusionar* es meter los cuerpos de los no terminales involucrados dentro del padre y borrarlos: quedan menos no terminales, pero el punto donde se decide *qué construcción era* pierde el nombre, y el analizador semántico pierde el lugar donde colgar el nodo del árbol. *Insertar un no terminal* es factorizar el prefijo y poner la disyuntiva restante en un no terminal nuevo y dedicado, cuyo nombre dice qué se está decidiendo: un no terminal más, pero se traduce a un método del analizador que **es** el punto de decisión semántica.

Se adopta siempre la segunda, con dos corolarios. **Un no terminal de decisión por contexto, aunque la forma coincida**: dos disyuntivas con la misma pinta pero distinto significado no se comparten. Y **compartir sí cuando es literalmente la misma construcción**: `<CuerpoDeMetodo>` se usa desde método, constructor y método **static** porque en los tres casos es lo mismo.

5.2 Eliminación de recursión a izquierda

El esquema general:

$$A \to A\alpha_1 \mid \cdots \mid A\alpha_n \mid \beta_1 \mid \cdots \mid \beta_m \quad \Longrightarrow \quad \begin{cases} A \to \beta_1 R \mid \cdots \mid \beta_m R \\ R \to \alpha_1 R \mid \cdots \mid \alpha_n R \mid \epsilon \end{cases}$$

5.2.1 R1 — `<ListaArgsFormales>`

1. Original

```
<ListaArgsFormales> ::= <ArgFormal>
<ListaArgsFormales> ::= <ListaArgsFormales> , <ArgFormal>
```

2. Defecto Recursión a izquierda directa. Ambas alternativas tienen Primeros(`<ArgFormal>`) = Primeros(`<Tipo>`) = {**boolean**, **char**, **int**, **idClase**, **idGen**}. Un analizador recursivo que intentara la segunda producción se llamaría a sí mismo sin haber consumido ningún token: recursión infinita.

3. Transformación Esquema directo, con α_1 = `,` `<ArgFormal>` y β_1 = `<ArgFormal>`.

4. Resultado

<ListaArgsFormales> ::= <ArgFormal> <RestoListaArgsFormales>

<RestoListaArgsFormales> ::= , <ArgFormal> <RestoListaArgsFormales> | ϵ

{,} contra {ϵ}: disjuntos.

5.2.2 R2 — <ExpresionCompuesta>

1. Original

<ExpresionCompuesta> ::= <ExpresionCompuesta> <OperadorBinario>
<ExpresionBasica>

<ExpresionCompuesta> ::= <ExpresionBasica>

2. Defecto Recursión a izquierda directa; es el caso canónico $E \to E + T \mid T$.

3. Transformación Esquema directo.

4. Resultado

<ExpresionCompuesta> ::= <ExpresionBasica> <RestoExpresionCompuesta>

<RestoExpresionCompuesta> ::= <OperadorBinario> <ExpresionBasica>
<RestoExpresionCompuesta> | ϵ

Los trece operadores binarios contra {ϵ}: disjuntos. Como se señaló, la gramática original ya era plana en precedencia, así que la transformación no pierde nada.

5.2.3 R3 — <Referencia>, y el conflicto que la transformación genera

1. Original

<Referencia> ::= <Primario>

<Referencia> ::= <Referencia> <VarEncadenada>

<Referencia> ::= <Referencia> <MetodoEncadenado>

<Referencia> ::= <Referencia> <AccesoArreglo>

2. Defecto Recursión a izquierda con *tres* alternativas recursivas: las cuatro producciones tienen los Primeros de <Primario>, lo que da los seis pares en conflicto.

3. Transformación Esquema directo con β_1 = <Primario> y los tres eslabones como α_i.

4. Resultado intermedio, con un defecto nuevo

<Referencia> ::= <Primario> <RestoReferencia>

<RestoReferencia> ::= <VarEncadenada> <RestoReferencia> |
<MetodoEncadenado> <RestoReferencia> | <AccesoArreglo>
<RestoReferencia> | ϵ

Primeros(<VarEncadenada>) = Primeros(<MetodoEncadenado>) = {.}: **siguen chocando**. Es exactamente el fenómeno que obliga a hacer la recursión antes que la factorización.

5. Factorización del defecto nuevo Los dos comparten *todo* el prefijo y sólo se distinguen por si después viene un (. Se fusionan, y la disyuntiva va a un no terminal dedicado.

<Referencia> ::= <Primario> <RestoReferencia>

`<RestoReferencia>` ::= `<Encadenado>` `<RestoReferencia>` | `<AccesoArreglo>` `<RestoReferencia>` | ϵ

`<Encadenado>` ::= `.` idMetVar `<EncadenadoAtributoOMetodo>`

`<EncadenadoAtributoOMetodo>` ::= `<ArgsActuales>` | ϵ

{`.`}, {`[`}, {ϵ}: disjuntos.

Decisión — costo semántico de R3, declarado

Se pierde la distinción entre `<VarEncadenada>` (acceso a un atributo de otro objeto) y `<MetodoEncadenado>` (llamada encadenada). Es una pérdida **inevitable**: sintácticamente son indistinguibles hasta ver el token siguiente. Queda mitigada por el no terminal insertado: `<EncadenadoAtributoOMetodo>` *es* el punto donde el analizador decide cuál de las dos era, según haya derivado a vacío o no. El nombre lo dice.

5.3 Factorización a izquierda

$$X \to \alpha\beta_1 \mid \cdots \mid \alpha\beta_n \mid \gamma_1 \mid \cdots \mid \gamma_m \quad \Longrightarrow \quad \begin{cases} X \to \alpha R \mid \gamma_1 \mid \cdots \mid \gamma_m \\ R \to \beta_1 \mid \cdots \mid \beta_n \end{cases}$$

5.3.1 F1 — `<Expresion>`

1–2 Prefijo común α = `<ExpresionCompuesta>`: viendo `x`, el analizador no sabe si va a aparecer un `=`, y puede estar arbitrariamente lejos.

3–4. Resultado

`<Expresion>` ::= `<ExpresionCompuesta>` `<RestoExpresion>`

`<RestoExpresion>` ::= `<OperadorAsignacion>` `<ExpresionCompuesta>` | ϵ

Decisión — la transformación «natural» de F1 agranda el lenguaje

La versión que sale sola al escribirla, y que además queda más prolija, es `<RestoExpresion>` ::= `<OperadorAsignacion>` `<Expresion>` | ϵ. **Es incorrecta**: la regla original tiene `<ExpresionCompuesta>` a la derecha del `=`, o sea que la asignación no es asociativa y `a = b = c` no pertenece al lenguaje. Poner `<Expresion>` la volvería asociativa a derecha y aceptaría programas que la cátedra no define.

Una transformación sólo es válida si **preserva exactamente** el lenguaje generado. El mismo cuidado se aplicó en F4.

5.3.2 F2 — `<Sentencia>`: resolver la ambigüedad

1–2 No es un prefijo común sino la ambigüedad descripta en la sección 4: `<Asignacion>` y `<Llamada>` tienen cuerpos idénticos.

3. Transformación No hay factorización que lo arregle: hay que *elegir*. Se colapsan en una única alternativa, con un nombre que dice lo que efectivamente es.

4. Resultado

`<Sentencia>` ::= `;` | `<SentenciaExpresion>` `;` | `<VarLocal>` `;` | `<Return>` `;` | `<If>` | `<While>` | `<Bloque>`

`<SentenciaExpresion>` ::= `<Expresion>`

Decisión — distinguir asignación de llamada es trabajo del semántico

Colapsar las dos alternativas **no achica el lenguaje**: lo genera exactamente igual, con un árbol por sentencia en lugar de dos. Determinar si la expresión es una asignación válida, una llamada válida o ninguna de las dos — `5 = x;` atraviesa el análisis sintáctico — requiere conocer el significado de la expresión, y eso es análisis semántico.

5.3.3 F3 — `<If>`: factorizar, y después resolver el *dangling else*

1–2 Prefijo común larguísimo: **if (`<Expresion>`) `<Sentencia>`**. El analizador debería atravesar una condición y una sentencia entera antes de saber qué producción eligió.

3–4. Resultado

`<If>` ::= **if (** `<Expresion>` **)** `<Sentencia>` `<ElseOpcional>`

`<ElseOpcional>` ::= **else** `<Sentencia>` | ϵ

{**else**} contra {ϵ}: disjuntos.

5. Lo que la factorización no arregla La ambigüedad del *dangling else* sigue ahí, escondida en que `<ElseOpcional>` puede derivar a ϵ *y* tener un **else** en lo que le sigue.

Decisión — el *dangling else* se resuelve con una regla de implementación

Es la segunda de las dos soluciones posibles ante una ambigüedad: en lugar de reescribir la gramática, se agrega una regla en la implementación del analizador. El esquema de traducción toma la rama **else** apenas ve el token, y como el **if** que se está ejecutando en ese momento es el más interno, el **else** liga al **if** más cercano — **que es exactamente la regla de Java**. No hace falta tocar la gramática: alcanza con dejar constancia de que el orden de las ramas en el método de `<ElseOpcional>` es significativo.

5.3.4 F4 — `<DimensionesConTamanio>`

1–2 Prefijo común **[`<Expresion>`]**.

3. Transformación **Con cuidado**: la regla original exige *al menos una* dimensión. La versión `<DimensionesConTamanio>` ::= **[** `<Expresion>` **]** `<DimensionesConTamanio>` | ϵ permitiría cero y agrandaría el lenguaje. La parte repetible va a un no terminal opcional aparte.

4. Resultado

`<DimensionesConTamanio>` ::= **[** `<Expresion>` **]** `<DimensionesConTamanioOpcional>`

`<DimensionesConTamanioOpcional>` ::= `<DimensionesConTamanio>` | ϵ

5.3.5 F5 — `<ListaExps>`

Prefijo común `<Expresion>`. Se factoriza con la misma forma que R1, para que las dos listas de la gramática se lean igual.

`<ListaExps>` ::= `<Expresion>` `<RestoListaExps>`

`<RestoListaExps>` ::= **,** `<Expresion>` `<RestoListaExps>` | ϵ

5.3.6 F6 — <Miembro>: factorización profunda

1. Original Ver sección 5; las tres alternativas son <Atributo>, <Metodo> y <Constructor>.

2. Defecto Prefijo común **no directo**: mirando sólo las tres alternativas de <Miembro> no se ve nada compartido, pero al bajar a sus producciones aparece que <Atributo> y <Metodo> empiezan los dos con <Tipo>.

$$\text{Primeros}(\texttt{<Atributo>}) = \text{Primeros}(\texttt{<Tipo>}) = \{\textbf{boolean}, \textbf{char}, \textbf{int}, \textbf{idClase}, \textbf{idGen}\}$$
$$\text{Primeros}(\texttt{<Metodo>}) = \{\textbf{static}, \textbf{void}\} \cup \text{Primeros}(\texttt{<Tipo>})$$

Ante `int x` el analizador no sabe si sigue un **;** o un **(**, y eso está dos tokens más adelante.

3. Transformación Factorización **profunda**: subir las producciones de <Atributo> y <Metodo> hasta <Miembro>, expandir <ModificadorOpcional> y <TipoMetodo> para exponer el prefijo, y recién ahí hacer factor común. Expandiendo:

<Miembro> ::= <Tipo> **idMetVar ;**

<Miembro> ::= **static** <TipoMetodo> **idMetVar** <ArgsFormales> <Bloque>

<Miembro> ::= <Tipo> **idMetVar** <ArgsFormales> <Bloque>

<Miembro> ::= **void idMetVar** <ArgsFormales> <Bloque>

<Miembro> ::= **public idClase** <ArgsFormales> <Bloque>

Ahora el prefijo <Tipo> **idMetVar** sí es directo, entre la primera y la tercera.

4. Resultado

<Miembro> ::= <Tipo> **idMetVar** <AtributoOMetodo>

<Miembro> ::= **static** <TipoMetodo> <Metodo>

<Miembro> ::= **void** <Metodo>

<Miembro> ::= <Constructor>

<AtributoOMetodo> ::= **;** | <CuerpoDeMetodo>

<Metodo> ::= **idMetVar** <CuerpoDeMetodo>

<CuerpoDeMetodo> ::= <ArgsFormales> <Bloque>

<Constructor> ::= **public idClase** <CuerpoDeMetodo>

{**boolean**, **char**, **int**, **idClase**, **idGen**}, {**static**}, {**void**}, {**public**}: disjuntos. Y en <AtributoOMetodo>, {**;**} contra {**(**}: disjuntos.

<Constructor> **sobrevive como no terminal propio** porque **public** lo distingue solo. Se extraen además dos no terminales compartidos: <CuerpoDeMetodo> (argumentos formales más bloque), que es literalmente la misma construcción en método, método **static** y constructor; y <Metodo>, que comparten la rama **static** y la rama **void**, de modo que el analizador semántico tiene un único lugar donde construir el nodo de método para esas dos formas. La tercera forma — la de tipo de referencia — ya consumió el **idMetVar** al factorizar contra el atributo, y por eso llega por <AtributoOMetodo>.

5.3.7 F7 — <Primario>: el prefijo idMetVar

<AccesoVar> (**idMetVar**) y <LlamadaMetodo> (**idMetVar** <ArgsActuales>) comparten todo el prefijo. Misma situación que en R3 y misma resolución, con un no terminal de decisión **dedicado a este contexto**:

<AccesoVarOLlamada> ::= idMetVar <AccesoLlamada>

<AccesoLlamada> ::= <ArgsActuales> | ϵ

Decisión — un punto de decisión por contexto, aunque la forma coincida

Una versión anterior de esta transformación resolvía R3 y F7 con el *mismo* no terminal <ArgsActualesOpcional>, y se había justificado como una buena abstracción. Es al revés: su ϵ significa dos cosas distintas según quién lo llame — desde <Encadenado> era un <VarEncadenada>, acceso a un atributo de *otro* objeto; desde <AccesoVarOLlamada> era un <AccesoVar>, acceso a una variable local, un parámetro o un atributo propio. Son dos nodos distintos, con reglas de resolución de nombres distintas, y un único método del analizador tendría que preguntarse quién lo llamó para saber cuál construir. Se separaron en <EncadenadoAtributoOMetodo> y <AccesoLlamada>: cuesta un no terminal más y deja los cuatro no terminales originales recuperables desde el nombre del método que decide.

5.3.8 F8 — <Primario>: el prefijo new, a dos niveles

2. Defecto <CrearArreglo> y <LlamadaConstructor> comparten new, y no alcanza con factorizarlo porque <TipoBase> puede derivar a <TipoReferencia>: `new C` sigue sin decidirse. El token que desempata es el que viene *después* del tipo.

3–4. Resultado Se factoriza new, se expande <TipoBase> para separar los casos que sí se deciden con un token, y se factoriza una segunda vez el caso idClase.

<Creacion> ::= new <ArregloOInstancia>

<ArregloOInstancia> ::= <TipoPrimitivo> <DimensionesConTamanio>

<ArregloOInstancia> ::= idGen <DimensionesConTamanio>

<ArregloOInstancia> ::= idClase <TipoGenericoOpcional>
<ArregloOInstanciaDeClase>

<ArregloOInstanciaDeClase> ::= <DimensionesConTamanio> | <ArgsActuales>

`new int[3]` y `new T[3]` se resuelven en el primer token después de new; sólo `new C...` necesita el segundo nivel. La factorización se hizo **sólo donde hacía falta**.

5.4 Estado del núcleo

Con R1–R3 y F1–F8, el núcleo de la gramática — sin logros — es LL(1). El costo respecto de la base fue de **seis no terminales y ocho producciones**, concentrado en los <Resto...> y en los puntos de decisión insertados, que el esquema de traducción convierte en un `if/else` cada uno.

6 Extensiones por logro

Los nueve logros de la Etapa 2 que afectan al analizador, más el *carry-over* de los floats de la Etapa 1, incorporados a la única gramática del compilador. Mismo procedimiento de cuatro pasos que el núcleo, con una diferencia: acá los conflictos no vienen de la gramática de la cátedra, los crea el logro.

El criterio de ubicación es uno solo: **un logro agrega una capa o unas alternativas propias; no reescribe las reglas de al lado**. De ahí que ninguno se apoye en un no terminal que introdujo otro — cuando dos necesitan una construcción parecida, cada uno

lleva la suya (sección 6.2). Compartirla ahorraría dos renglones y le costaría al semántico desambiguar en ejecución lo que la gramática podía separar de entrada.

6.1 Visibilidad Mejorada E2

Defecto que crea **public** y **private** pasan a poder encabezar cualquier miembro. El **public** del `<Constructor>` original queda absorbido, el constructor se vuelve **idClase** `<CuerpoDeMetodo>` y **empieza a competir** con el atributo o método de tipo referencia por el token **idClase**: `C(){}` contra `C x;`.

Transformación Factorizar la visibilidad al frente y, adentro, el **idClase**. El desempate es el token que sigue: **(** es constructor, cualquier otro abre un tipo.

Sobre el orden: MiniJava no dice nada, así que manda Java y se aceptan `public static` y `static public`. Repetir un modificador no se acepta, y no hace falta chequearlo fuera de la gramática: alcanza con que la visibilidad aparezca a **un solo lado** del **static**.

```
<Miembro> ::= <Visibilidad> <MiembroTrasVisibilidad>
<Miembro> ::= static <VisibilidadOpcional> <TipoMetodo> <Metodo>
<Miembro> ::= <MiembroSinModificadores>
<MiembroTrasVisibilidad> ::= static <TipoMetodo> <Metodo> |
    <MiembroSinModificadores>
<Visibilidad> ::= public | private
<VisibilidadOpcional> ::= <Visibilidad> | ϵ
<MiembroSinModificadores> ::= void <Metodo>
<MiembroSinModificadores> ::= <TipoPrimitivo> <DimensionesOpcionales>
    idMetVar <AtributoOMetodo>
<MiembroSinModificadores> ::= idGen <DimensionesOpcionales> idMetVar
    <AtributoOMetodo>
<MiembroSinModificadores> ::= idClase <MiembroDesdeIdClase>
<MiembroDesdeIdClase> ::= <CuerpoDeMetodo>
<MiembroDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales>
    idMetVar <AtributoOMetodo>
```

Resultado En `<Miembro>`, tres conjuntos disjuntos: {**public**, **private**}, {**static**} y los Primeros de `<MiembroSinModificadores>`. En `<MiembroDesdeIdClase>`, {**(**} contra {<, **[**, **idMetVar**}.

static sigue aplicando sólo a métodos, como en MiniJava base. **protected** no se implementa: no figura en MiniJava.

Entrada	Veredicto
`public static void m(){}`	acepta
`static public void m(){}`	acepta
`private static int m(){}`	acepta
`static private int m(){}`	acepta
`public private void m(){}`	**rechaza**
`static static void m(){}`	**rechaza**
`public static private void m(){}`	**rechaza**

Decisión — en los métodos de interfaz se admite public y no private

El logro habla de «Métodos, Atributos y Constructores», y `<MetodoInterfaz>` es un no terminal aparte de `<Miembro>`: cae bajo la regla general, lo no especificado se comporta como en Java. Verificado con `javac`, en Java un método de interfaz **private exige cuerpo** —`private int m();` es *missing method body*, y `private int m(){...}` compila—. En MiniJava no hay forma de escribir ese cuerpo: `<MetodoInterfaz>` termina en **;** y ningún logro lo cambia. Un **private** ahí sería siempre la forma que Java rechaza.

De ahí la asimetría, que es una consecuencia y no una preferencia:

```
<MetodoInterfaz> ::= <PublicOpcional> <TipoMetodo> idMetVar <ArgsFormales>
    ;
<PublicOpcional> ::= public | ϵ
```

6.2 Atributos Inicializados y Variables locales Clásicas E2

Los dos agregan un **=** `<ExpresionCompuesta>` opcional. La tentación es compartir un único no terminal; está mal por el criterio de la sección 6, porque son construcciones distintas y el semántico las trata distinto. Van dos dedicados con el mismo cuerpo, `<InicializacionDeAtributo>` e `<InicializacionDeDeclarador>`.

Defecto que crea *varclasica* Una declaración local pasa a poder empezar con **idClase** (`C x;`), y una sentencia-expresión también (`C.m();`, por `<LlamadaMetodoEstatico>`). Chocan.

Transformación Factorizar el **idClase** en `<Sentencia>`: **.** significa expresión, cualquier otro token significa declaración.

```
<Sentencia> ::= <ExpresionSinIdClase> ;
<Sentencia> ::= <TipoPrimitivo> <DimensionesOpcionales>
    <ListaDeclaradores> ;
<Sentencia> ::= idGen <DimensionesOpcionales> <ListaDeclaradores> ;
<Sentencia> ::= idClase <SentenciaDesdeIdClase>
<SentenciaDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales>
    <ListaDeclaradores> ;
<SentenciaDesdeIdClase> ::= . idMetVar <ArgsActuales>
    <ColaDeExpresionDesdePrimario> ;
```

La rama de expresión tiene que retomar una expresión *ya empezada*. Escribirla a mano duplicaría media jerarquía; en su lugar, tres no terminales que capturan las colas compartidas:

```
<ExpresionSinIdClase> ::= <OperadorUnario> <Operando> <ColaTrasOperando>
<ExpresionSinIdClase> ::= <Primitivo> <ColaTrasOperando>
<ExpresionSinIdClase> ::= <PrimarioSinIdClase>
    <ColaDeExpresionDesdePrimario>
<ColaTrasOperando> ::= <RestoExpresionCompuesta> <RestoCondicional>
    <RestoExpresion>
<ColaDeExpresionDesdePrimario> ::= <RestoReferencia> <PosfijoOpcional>
    <ColaTrasOperando>
<Primario> ::= <LlamadaMetodoEstatico> | <PrimarioSinIdClase>
```

`<LlamadaMetodoEstatico>` es la única forma de `<Primario>` que empieza con **idClase**, así que la partición es exacta. **Sólo se duplican las cabezas**: las colas se comparten, de modo que un cambio futuro en las expresiones no hay que espejarlo.

Decisión — alternativa descartada: dos tokens de anticipación

El mismo conflicto se resuelve mirando si después del **idClase** viene un **.**, con cero no terminales nuevos. Se descartó porque despega la implementación del esquema de Clase 5, que es de un único token y es la garantía de correctitud que se está siguiendo.

6.3 Fors! E2

Un `<For>` nuevo como alternativa de `<Sentencia>`. El encabezado tiene el mismo problema de **idClase** que la sentencia y se resuelve igual, reutilizando `<ExpresionSinIdClase>` y `<ColaDeExpresionDesdePrimario>`. El desempate entre el **for** estándar y el de iteradores es de un solo token y está justo después del declarador: **:** es de iteradores, **=** o **;** es estándar.

```
<For> ::= for ( <EncabezadoFor> ) <Sentencia>
<EncabezadoFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional>
<EncabezadoFor> ::= var idMetVar <EncabezadoForTrasVar>
<EncabezadoFor> ::= <TipoPrimitivo> <DimensionesOpcionales> idMetVar
    <EncabezadoForTrasDeclarador>
<EncabezadoFor> ::= idGen <DimensionesOpcionales> idMetVar
    <EncabezadoForTrasDeclarador>
<EncabezadoFor> ::= idClase <EncabezadoForDesdeIdClase>
<EncabezadoFor> ::= <ExpresionSinIdClase> <ClausulasFinalesFor>
<EncabezadoForTrasDeclarador> ::= : <Expresion>
<EncabezadoForTrasDeclarador> ::= <InicializacionDeDeclaradorFor>
    <ClausulasFinalesFor>
<ClausulasFinalesFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional>
```

`<ClausulasFinalesFor>` con una sola `<ExpresionOpcional>` por separador es la lectura literal de «se restringe a una sola sentencia o expresión por separador».

6.4 Operador Ternario E2

Una capa nueva entre la asignación y los operadores binarios, con la precedencia de Java.

`<Expresion>` ::= `<ExpresionCondicional>` `<RestoExpresion>`

`<RestoExpresion>` ::= `<OperadorAsignacion>` `<ExpresionCondicional>` | ϵ

`<ExpresionCondicional>` ::= `<ExpresionCompuesta>` `<RestoCondicional>`

`<RestoCondicional>` ::= `?` `<ExpresionCondicional>` `:` `<ExpresionCondicional>` | ϵ

Asociativo a **derecha** — `a ? b : c ? d : e` es `a ? b : (c ? d : e)` — porque la rama del `:` vuelve a `<ExpresionCondicional>`.

Decisión — dónde alcanza el ternario

La gramática original usa `<ExpresionCompuesta>` como lado derecho de un `=` en los *seis* lugares donde aparece uno — la asignación y los cinco inicializadores —, para impedir `a = b = c`. Insertar la capa del ternario *por encima* dejaría a esos seis sin el operador, y `var x = c ? a : b;` sería inválido aunque `x = c ? a : b;` no lo fuera. Los seis pasan entonces a `<ExpresionCondicional>`, un escalón más arriba: sigue sin admitir `=`, pero admite el ternario. La capa no le saca nada al lenguaje base, sólo le agrega.

Requiere el token `?`, que no existe en el léxico de la Etapa 1.

6.5 Operadores Posifijos E2

`<Operando>` ::= `<Primitivo>` | `<Referencia>` `<PosfijoOpcional>`

`<PosfijoOpcional>` ::= `++` | `--` | ϵ

El posfijo va sobre `<Referencia>` y **no** sobre `<Operando>`: `5++` no tiene sentido. Misma técnica de desactivación que el ternario. Es el único logro que **no** requiere tocar el léxico: `++` y `--` ya son tokens desde la Etapa 1 y no tenían dónde usarse.

6.6 MiniLambdas E2

Defecto que crea Dos.

Dónde vive la lambda. El JLS la pone como alternativa de *Expression*, el nivel más alto, nunca como *Primary*. Ponerla en `<Primario>` —la ranura del operando— haría válidos `1 + x -> x`, `- x -> x` y `(x -> x).m()`, que `javac` rechaza con *«lambda expression not expected here»* (sección 4.3).

La anticipación. El `(` abre tanto una expresión parentizada como la lista de parámetros de una lambda, y `idMetVar` tanto un acceso a variable como una lambda de un parámetro. `(a, b, c) -> x` contra `(a + b)` parece exigir escanear hasta el `)`.

Análisis No lo exige. Los casos se separan solos:

Entrada	Decisión	Tokens
`( )`	sólo puede ser lambda de cero parámetros	1 (el `)`)
`(` *e* `,` ...	sólo puede ser lambda: no hay operador coma	1 (la `,`)
`(` *e* `)`	lo decide el token *posterior* al `)`	1 (el `->`)
`idMetVar ->`	lo decide el token posterior al identificador	1 (el `->`)

La clave es que el -> viene *inmediatamente* después del), así que en el momento de decidir sigue siendo un token de anticipación.

Decisión — el paréntesis se resuelve en la gramática, no en la implementación

> **Decidir si un (abre una expresión parentizada o la lista de parámetros de una lambda es trabajo de la gramática, y lo hace entera.** El analizador no adivina, no vuelve atrás y no mira más de un token: elige producción igual que en cualquier otro punto del Esquema Simple.

Los cuatro no terminales que lo resuelven son `<ParentizadaOLambda>`, `<ContenidoDeParentesis>`, `<RestoDeParentesis>` y `<FinDeParentizadaOLambda>`, y cada uno decide con los Primeros de sus alternativas, que son disjuntos. No hay *backtracking*, ni anticipación de dos tokens, ni un marcador que el analizador consulte: si hiciera falta algo de eso, la gramática no sería LL(1) y el verificador lo diría.

Lo que la gramática **no** puede expresar son dos restricciones que no dependen de qué producción tomar, sino de qué se leyó una vez tomada, y ésas sí las cierra el analizador:

Restricción	Por qué no es expresable
El primer parámetro es un identificador sin tipo —`(a + 1) -> a` se rechaza—	Hasta ver el -> el contenido del paréntesis es una `<Expresion>` en los dos casos, así que *la misma producción* genera la lambda válida y la inválida. El analizador lee la expresión y recuerda si consumió exactamente un **idMetVar**. Del segundo parámetro en adelante sí lo impone la gramática, porque ahí ya se sabe que es una lista.
Una lambda parentizada no lleva cola —`(x -> y).m()` se rechaza—	`<ContenidoDeParentesis>` tiene que admitir una lambda, porque `f((x -> y))` es válido. Distinguir los dos casos en la gramática exigiría clonar la cadena entera de colas en una variante «el contenido fue una lambda»: unos siete no terminales nuevos, que es justamente lo que Clase 4, slide 40, desaconseja.

O sea: la *ambigüedad* está resuelta en la gramática y la *permisividad que queda* la cierra el analizador, que es la segunda de las dos vías que autoriza Clase 4, slide 49 — la misma con la que se resuelve el *dangling else*.

Resultado Un nivel nuevo por encima de `<ExpresionCondicional>` que admite lambda y no admite =, que es exactamente el *LambdaExpression* / *ConditionalExpression* del JLS, y donde se factorizan las dos cabezas ambiguas:

```
<Expresion> ::= <ExpresionCondicionalOLambda> <RestoExpresion>
<ExpresionSinLambda> ::= <ExpresionCondicional> <RestoExpresion>
<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCondicionalOLambda> |
    ϵ
<ExpresionCondicionalOLambda> ::= <OperadorUnario> <Operando>
    <ColaTrasOperandoSinAsignacion>
<ExpresionCondicionalOLambda> ::= <Primitivo>
    <ColaTrasOperandoSinAsignacion>
```

```
<ExpresionCondicionalOLambda> ::= <ExpresionDesdePrimarioOLambda>
<ExpresionDesdePrimarioOLambda> ::= idMetVar <AccesoLlamadaOLambda>
<ExpresionDesdePrimarioOLambda> ::= <ParentizadaOLambda>
<ExpresionDesdePrimarioOLambda> ::= idClase . idMetVar <ArgsActuales>
    <ColaDesdePrimarioSinAsignacion>
<ExpresionDesdePrimarioOLambda> ::= this <ColaDesdePrimarioSinAsignacion>
<ExpresionDesdePrimarioOLambda> ::= stringLiteral
    <ColaDesdePrimarioSinAsignacion>
<ExpresionDesdePrimarioOLambda> ::= <Creacion>
    <ColaDesdePrimarioSinAsignacion>
<AccesoLlamadaOLambda> ::= -> <CuerpoLambda> | <AccesoLlamada>
    <ColaDesdePrimarioSinAsignacion>
<ParentizadaOLambda> ::= ( <ContenidoDeParentesis>
<ContenidoDeParentesis> ::= ) -> <CuerpoLambda> | <Expresion>
    <RestoDeParentesis>
<RestoDeParentesis> ::= ) <FinDeParentizadaOLambda>
<RestoDeParentesis> ::= , <ListaParametrosLambda> ) -> <CuerpoLambda>
<FinDeParentizadaOLambda> ::= -> <CuerpoLambda> |
    <ColaDesdePrimarioSinAsignacion>
<CuerpoLambda> ::= <Expresion>
```

Ningún `<Primario>` puede ya ser una lambda: `<Parentizada> ::= ( <ExpresionSinLambda> )`, y `<AccesoVarOLlamada>` perdió la alternativa del `->`. Los seis lugares donde el JLS sí admite una lambda —argumento, índice de arreglo, las dos ramas del ternario, lado derecho de un `=` y cuerpo de otra lambda— la siguen admitiendo, porque los seis pasan por `<Expresion>` o por `<ExpresionCondicionalOLambda>`. `<CuerpoLambda> ::= <Expresion>` es la lectura literal de «el cuerpo debe estar formado por una única expresión, no se permiten cuerpos delimitados por llaves»: como no hay alternativa con `{`, la gramática **rechaza** `x -> { ...}` sin ningún chequeo adicional.

Requiere el token `->`, que no existe en el léxico de la Etapa 1.

Decisión — permisividad declarada de las lambdas

En `( a , b ) -> x` el `a` se deriva como `<Expresion>`, no como parámetro, de modo que `( a + 1 , b ) -> x` **atraviesa la gramática**: rechazarlo exige saber que esa expresión tiene que ser un identificador suelto, y eso una gramática libre de contexto no lo puede decir. Es la misma clase de permisividad que la gramática original ya tiene con `5 = x;`.

La diferencia está en qué pasa después. El `5 = x;` queda para el análisis semántico, y el compilador lo acepta. El parámetro que no es un identificador sin tipo, en cambio, **lo rechaza este analizador**: no hace falta la tabla de símbolos, alcanza con lo que acaba de leer —la decisión que sigue lo desarrolla, junto con el `this` y la captura, y la sección 1.2 fija el criterio con el que se decide caso por caso—. Lo permisiva que es la gramática y lo permisivo que es el compilador son dos cosas distintas.

Hay una segunda, del mismo origen. `<ContenidoDeParentesis> ::= <Expresion> <RestoDeParentesis>`, y esa `<Expresion>` puede ser una lambda —`f((x -> x))` es

válido en Java—. Pero `(x -> x).m()` no lo es: parentizarla no la convierte en operando. Distinguirlos en la gramática exigiría clonar toda la cadena de colas en su variante «el contenido del paréntesis fue una lambda», unos siete no terminales nuevos, así que lo cierra el analizador con lo que acaba de leer.

Lo que verifica es si la **raíz** de esa expresión es una lambda, no si se vio una lambda en algún lado: `(a = x -> x).m()` es una asignación y `(a ? b : x -> x).m()` un ternario, y Java acepta la forma de los dos. `javac` igual rechaza el segundo, pero por la regla de *poly expressions* del JLS 15.25, que depende de la clasificación de la expresión y de su tipo destino —un chequeo del analizador semántico, no de la forma—. Queda declarado.

Decisión — el this, la captura y el parámetro repetido van al analizador semántico

Las otras tres cláusulas del logro — «no se permite la captura de variables locales, parámetros, ni el uso de **this**», más el requisito implícito de que dos parámetros no se llamen igual — **no las controla esta etapa**. Es indicación del profesor, y el motivo técnico coincide con el criterio de la sección 1.2: ninguna de las tres es decidible con información local.

Dentro del cuerpo de una lambda, `x -> campo` — un atributo, o sea un **this** implícito — y `x -> n` — una variable de afuera, o sea una captura — son **la misma forma**: un **idMetVar** suelto que no está entre los parámetros. Separarlas exige resolver el nombre contra su declaración, que es exactamente la tabla de símbolos de la Etapa 3. El caso extremo es la llamada sin punto: `x -> ayuda(x)` es un **this** implícito si `ayuda` es un método de instancia, y no lo es si es **static**.

El **this** *explícito* sí sería decidible con un booleano, pero controlarlo solo no aporta: dejaría rechazado `x -> this.campo` y aceptado `x -> campo`, que es el mismo error escrito de otra forma.

Esto simplificó el analizador de manera apreciable. Se borró la clase que sostenía los cuatro chequeos, y con ella su pieza más intrincada: como leer el contenido de un paréntesis lo hacía pasar por el chequeo de captura *antes* de saber si era una lista de parámetros, en `x -> (y) -> y` el `y` se habría reportado como captura de `x`; hacía falta anotar los nombres «en duda» en una pila y resolverlos recién al cerrar el paréntesis. Sin chequeo de captura, esa maquinaria no tiene motivo.

Qué chequea y qué no, entonces Ésta es la frontera completa del logro:

Cláusula	Dónde se verifica
forma `params -> expr`	gramática
parámetros sin tipo	gramática
cuerpo de una única expresión, sin llaves	gramática
dónde puede aparecer una lambda	gramática: `<Expresion>`, no `<Primario>`
el parámetro es un identificador sin tipo	analizador sintáctico
una lambda parentizada no lleva cola	analizador sintáctico
no **this**, explícito ni implícito	**analizador semántico**
no captura de variables locales ni parámetros	**analizador semántico**
parámetros sin nombre repetido	**analizador semántico**

Con ejemplos concretos, que es lo que importa a la hora de leer la salida del compilador:

Entrada	Veredicto	Por qué
`x -> x`	acepta	
`() -> arrancar()`	acepta	
`(a, b) -> a * b`	acepta	
`x -> x.a.b.c`	acepta	
`f((x -> x))`	acepta	la lambda parentizada es toda la expresión
`x -> this.campo`	acepta	el **this** lo controla el semántico
`x -> x + n`	acepta	si `n` es captura o campo no se sabe acá
`x -> y -> x + y`	acepta	la interna usa `x`: lo ve el semántico
`(a, a) -> a`	acepta	el nombre repetido lo controla el semántico
`x -> { return x; }`	**rechaza**	el cuerpo no admite llaves
`(int x) -> x`	**rechaza**	el parámetro no lleva tipo
`(a + 1) -> a`	**rechaza**	el parámetro no es un identificador sin tipo
`((a)) -> a`	**rechaza**	parentizar de más no es parentizar el parámetro
`1 + x -> x`	**rechaza**	la lambda no es operando
`(x -> x).m()`	**rechaza**	parentizarla no la convierte en operando
`(x -> x) + 1`	**rechaza**	ídem

6.7 Inicializacion de Arreglos E2

El logro dice *«permite la inicialización de Arreglos al momento de su construcción como en Java. Sólo se permite como parte de una expresión de creación utilizando **new**»*. La única restricción que enuncia es *dónde*, no cuántas dimensiones. Pero hay un choque que conviene dejar escrito, porque no tiene salida limpia.

El choque MiniJava base define la creación de un arreglo así:

```
<CrearArreglo> ::= new <TipoBase> <DimensionesConTamanio>
<DimensionesConTamanio> ::= [ <Expresion> ] <DimensionesConTamanio> | [
    <Expresion> ]
```

Toda dimensión lleva su tamaño, y el corchete vacío no existe. Java exige exactamente lo contrario cuando hay inicializador. Verificado con `javac`:

En Java	Resultado
`new int[]{1,2,3}`	compila
`new int[][]{{1,2},{3,4}}`	compila
`new int[2]{1,2}`	**error**: *array creation with both dimension expression and initialization is illegal*

O sea que no se puede a la vez respetar la forma de las dimensiones que MiniJava ya tenía definida y el «como en Java» del enunciado.

Decisión — gana MiniJava base: la dimensión conserva su tamaño

Se deja `<DimensionesConTamanio>` tal cual la escribió la cátedra y el inicializador se le cuelga al final, opcional:

```
<CreacionDeArreglo> ::= <DimensionesConTamanio>
    <InicializadorDeArregloOpcional>
<DimensionesConTamanio> ::= [ <Expresion> ] <DimensionesConTamanioOpcional>
<DimensionesConTamanioOpcional> ::= <DimensionesConTamanio> | ϵ
<InicializadorDeArregloOpcional> ::= <InicializadorDeArreglo> | ϵ
<InicializadorDeArreglo> ::= { <ListaElementosOpcional> }
<ElementoInicializador> ::= <Expresion> | <InicializadorDeArreglo>
```

El criterio general del proyecto — lo que MiniJava no define se comporta como en Java — **no aplica acá**, porque MiniJava sí define la forma de las dimensiones. Adoptar la de Java sería ampliar el lenguaje en una construcción ya especificada. La divergencia queda aislada en una sola producción, `<CreacionDeArreglo>`, por si hubiera que revertirla.

Entrada	Veredicto
`new int[3]`	acepta — la forma de siempre
`new int[3][4]`	acepta
`new int[3]{1,2,3}`	acepta
`new int[2][2]{{1,2},{3,4}}`	acepta — el inicializador es anidable
`new int[0]{}`	acepta
`new int[]{1,2,3}`	**rechaza** — no hay corchete vacío
`new int[]`	**rechaza**

Resultado Primeros(`<Expresion>`) no contiene `{`, así que las dos alternativas de `<ElementoInicializador>` son disjuntas, y `{` tampoco puede seguir a una expresión de

creación en ningún contexto, así que `<InicializadorDeArregloOpcional>` no entra en conflicto con su ϵ. La restricción de que el inicializador sólo valga como parte de un **new** queda impuesta por la gramática: `<InicializadorDeArreglo>` sólo es alcanzable desde `<Creacion>`.

6.8 Genericidad Avanzada E2

Dos cosas distintas. El **anidamiento** sale de una sola recursión:

`<ArgumentoDeTipo>` ::= **idGen** | **idClase** `<TipoGenericoOpcional>`

Sin esa recursión la segunda alternativa sería **idClase** a secas. (`List<List<Persona>>` cierra con dos `>` separados: el léxico no define ningún token `>>`, así que la regla de *maximal munch* entrega dos `>` y no hay nada que resolver del lado sintáctico.)

El **diamante** exige más cuidado, porque el enunciado dice que «sólo puede utilizarse al instanciar una Clase Genérica». Se impone **por gramática**, separando el genérico en posición de tipo del genérico bajo **new**:

`<TipoGenericoOpcional>` ::= `<` `<ArgumentoDeTipo>` `>` | ϵ

`<CreacionDesdeIdClase>` ::= `<` `<GenericoDeCreacion>` | `<CreacionDeArreglo>` | `<ArgsActuales>`

`<GenericoDeCreacion>` ::= `>` `<ArgsActuales>`

`<GenericoDeCreacion>` ::= `<ArgumentoDeTipo>` `>` `<ArregloOInstanciaDeClase>`

La primera alternativa de `<GenericoDeCreacion>` *es* el diamante, y va sólo hacia `<ArgsActuales>`. Resultado:

Entrada	Veredicto
`List<Persona> x;`	acepta
`List<List<Persona>> x;`	acepta
`new List<>()`	acepta
`new List<Persona>[3]`	acepta
`List<> x;`	**rechaza** (diamante en posición de tipo)
`new List<>[3]`	**rechaza** (diamante con arreglo)

Decisión — compartir un no terminal puede volverse incorrecto más adelante

En la gramática original, `<TipoGenericoOpcional>` se usa desde `<TipoReferencia>` y desde `<LlamadaConstructor>`, y compartirlo es correcto: son la misma construcción. **Deja de serlo con este logro**, porque el diamante vale en una posición y no en la otra. Compartiéndolo habría que aceptar `List<> x;` y rechazarlo después en el análisis semántico. Separarlos cuesta un no terminal y mueve la restricción al lugar donde se puede verificar sin conocer el significado de nada.

6.9 Floats (*carry-over* de la Etapa 1)

`<TipoPrimitivo>` ::= **boolean** | **char** | **int** | **float**

`<Primitivo>` ::= **true** | **false** | **intLiteral** | **charLiteral** | **floatLiteral** | **null**

Dos alternativas nuevas, cero conflictos. Sin ellas, el logro de floats de la Etapa 1 queda inutilizable en la Etapa 2: el léxico emite **float** y **floatLiteral** y el analizador sintáctico los rechazaría.

6.10 Impacto sobre el analizador léxico

Cuatro de los logros exigen elementos léxicos que **no figuran en *Aspectos Léxicos de* MiniJava**: la lista de palabras clave de §2.1 llega hasta **false** y la de operadores de §2.4.2 hasta ++ y --.

Elemento	Logro	Estado en la Etapa 1
for	Fors! E2	se reconoce como **idMetVar**
private	Visibilidad Mejorada E2	se reconoce como **idMetVar**
?	Operador Ternario E2	es un error léxico
->	MiniLambdas E2	se reconoce como - seguido de >

Se incorporan al analizador léxico **sin condición**, así que la columna de la derecha describe lo que pasaba en la Etapa 1 y no una configuración alcanzable hoy. El cambio no toca el patrón State ni el driver del autómata: `EstadoIdMV` reconoce **for** y **private**, `EstadoInicial` el **?**, y `EstadoOperadorConSufijoOpcional` el ->. Los tokens **:**, ++ y --, en cambio, ya existían.

7 Gramática final

La gramática completa resultante, en la misma notación del documento que publica la cátedra. Es la única gramática del compilador y no lleva ninguna marca: todas las producciones existen siempre.

Se **genera** desde el mismo archivo que verifica la herramienta de la sección 11, así que el informe y lo que se comprueba no pueden divergir.

<Inicial> ::= <ListaClases> **eof**

<ListaClases> ::= <Clase> <ListaClases> | <Interfaz> <ListaClases> | ϵ

<Clase> ::= **class idClase** <GenericidadOpcional> <HerenciaOpcional> **{**
    <ListaMiembros> **}**

<Interfaz> ::= **interface idClase** <GenericidadOpcional> <ExtensionOpcional> **{**
    <ListaMetodosInterfaz> **}**

Una Clase o Interfaz sigue limitada a UN único parámetro de tipo, también con la genericidad avanzada.

<GenericidadOpcional> ::= **< idGen >** | ϵ

<HerenciaOpcional> ::= **extends** <TipoReferencia> | **implements**
    <TipoReferencia> | ϵ

<ExtensionOpcional> ::= **extends** <TipoReferencia> | ϵ

<ListaMiembros> ::= <Miembro> <ListaMiembros> | ϵ

<ListaMetodosInterfaz> ::= <MetodoInterfaz> <ListaMetodosInterfaz> | ϵ

La visibilidad se factoriza al frente de los tres tipos de miembro. Eso hace que el `public` del <Constructor> original quede absorbido por `<Visibilidad>`, y que <Constructor> pase a competir con `<Tipo>` por el token idClase: `C(){}` (constructor) vs `C x;` (atributo de tipo C).

`<Miembro>` ::= `<Visibilidad>` `<MiembroTrasVisibilidad>`

`<Miembro>` ::= **static** `<VisibilidadOpcional>` `<TipoMetodo>` `<Metodo>`

`<Miembro>` ::= `<MiembroSinModificadores>`

Los dos ordenes valen, como en Java: `public static` y `static public`. Que la visibilidad aparezca a un solo lado de `static` es lo que impide repetir el modificador —`public private void m(){}` no deriva—, sin necesitar un chequeo fuera de la gramatica.

`<MiembroTrasVisibilidad>` ::= **static** `<TipoMetodo>` `<Metodo>`

`<MiembroTrasVisibilidad>` ::= `<MiembroSinModificadores>`

`<Visibilidad>` ::= **public** | **private**

`<VisibilidadOpcional>` ::= `<Visibilidad>` | ϵ

`<MiembroSinModificadores>` ::= **void** `<Metodo>`

`<MiembroSinModificadores>` ::= `<TipoPrimitivo>` `<DimensionesOpcionales>` **idMetVar** `<AtributoOMetodo>`

`<MiembroSinModificadores>` ::= **idGen** `<DimensionesOpcionales>` **idMetVar** `<AtributoOMetodo>`

`<MiembroSinModificadores>` ::= **idClase** `<MiembroDesdeIdClase>`

El NT insertado que decide entre constructor y miembro con tipo de referencia. Con visibilidad el constructor ya no lleva `public` propio, así que compite con el atributo/método de tipo referencia por el token idClase.

`<MiembroDesdeIdClase>` ::= `<CuerpoDeMetodo>`

`<MiembroDesdeIdClase>` ::= `<TipoGenericoOpcional>` `<DimensionesOpcionales>` **idMetVar** `<AtributoOMetodo>`

El NT insertado que decide entre atributo y método.

`<AtributoOMetodo>` ::= `<InicializacionDeAtributo>` **;**

`<AtributoOMetodo>` ::= `<CuerpoDeMetodo>`

NT dedicado y no compartido con la inicialización de una variable local: son construcciones distintas y el semántico las trata distinto.

`<InicializacionDeAtributo>` ::= **=** `<ExpresionCondicionalOLambda>`

`<InicializacionDeAtributo>` ::= ϵ

`<Metodo>` sobrevive como no terminal propio y lo comparten la rama static y la rama void, así que el semántico tiene UN solo lugar donde construir el nodo de método para esas dos formas. La tercera forma (tipo de referencia) ya consumió el idMetVar al factorizar contra el atributo, y llega por AtributoOMetodo.

`<Metodo>` ::= **idMetVar** `<CuerpoDeMetodo>`

Se comparte a propósito: es la misma construcción en método, método static y constructor.

`<CuerpoDeMetodo>` ::= `<ArgsFormales>` `<Bloque>`

Los métodos de interfaz admiten `public` pero no `private`, que es lo que hace Java: verificado con javac, un método de interfaz `private` exige cuerpo («missing method body, or declare abstract»), y MiniJava no tiene forma de escribir ese cuerpo —`<MetodoInterfaz>` termina en `;` en la especificación original—. O sea que un `private` acá sería siempre la forma que Java rechaza.

<MetodoInterfaz> ::= <PublicOpcional> <TipoMetodo> **idMetVar** <ArgsFormales> **;**

<PublicOpcional> ::= **public**

<PublicOpcional> ::= ϵ

<TipoMetodo> ::= <Tipo> | **void**

<Tipo> ::= <TipoBase> <DimensionesOpcionales>

<TipoBase> ::= <TipoPrimitivo> | <TipoReferencia> | **idGen**

<DimensionesOpcionales> ::= **[ ]** <DimensionesOpcionales> | ϵ

<TipoReferencia> ::= **idClase** <TipoGenericoOpcional>

`float` es token del léxico desde la Etapa 1 pero no estaba en la gramática; sin esta alternativa el `float` del léxico queda inutilizable.

<TipoPrimitivo> ::= **boolean** | **char** | **int**

<TipoPrimitivo> ::= **float**

En POSICIÓN DE TIPO no se admite el diamante: `List<> x;` es inválido.

<TipoGenericoOpcional> ::= **<** <ArgumentoDeTipo> **>** | ϵ

la recursión por <TipoGenericoOpcional> es la que habilita el anidamiento `List<List<Persona>>`.

<ArgumentoDeTipo> ::= **idGen**

<ArgumentoDeTipo> ::= **idClase** <TipoGenericoOpcional>

<ArgsFormales> ::= **(** <ListaArgsFormalesOpcional> **)**

<ListaArgsFormalesOpcional> ::= <ListaArgsFormales> | ϵ

<ListaArgsFormales> ::= <ArgFormal> <RestoListaArgsFormales>

<RestoListaArgsFormales> ::= **,** <ArgFormal> <RestoListaArgsFormales> | ϵ

<ArgFormal> ::= <Tipo> **idMetVar**

Una declaración local puede empezar con idClase (`C x;`), y una sentencia-expresión también (`C.m();`). Se factoriza ese idClase y el desempate lo hace el token siguiente: `.` es expresión, cualquier otro es declaración. La rama de expresión reusa <ColaDeExpresionDesdePrimario> para no duplicar la jerarquía de expresiones.

<Bloque> ::= **{** <ListaSentencias> **}**

<ListaSentencias> ::= <Sentencia> <ListaSentencias> | ϵ

<Sentencia> ::= **;**

<Sentencia> ::= <VarLocal> **;**

<Sentencia> ::= <Return> **;**

<Sentencia> ::= <If>

<Sentencia> ::= <While>

<Sentencia> ::= <Bloque>

<Sentencia> ::= <For>

Sin variables locales clásicas, una sentencia-expresión puede empezar con idClase sin competir con nada: es la regla original.

<Sentencia> ::= <ExpresionSinIdClase> **;**

<Sentencia> ::= <TipoPrimitivo> <DimensionesOpcionales> <ListaDeclaradores> **;**

<Sentencia> ::= idGen <DimensionesOpcionales> <ListaDeclaradores> ;

<Sentencia> ::= idClase <SentenciaDesdeIdClase>

<SentenciaDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales> <ListaDeclaradores> ;

<SentenciaDesdeIdClase> ::= . idMetVar <ArgsActuales> <ColaDeExpresionDesdePrimario> ;

<VarLocal> ::= var idMetVar = <ExpresionCondicionalOLambda>

Varias variables por sentencia, cada una con inicialización opcional: `int x,y,z = 10;`

<ListaDeclaradores> ::= idMetVar <InicializacionDeDeclarador> <RestoListaDeclaradores>

<RestoListaDeclaradores> ::= , idMetVar <InicializacionDeDeclarador> <RestoListaDeclaradores>

<RestoListaDeclaradores> ::= ϵ

<InicializacionDeDeclarador> ::= = <ExpresionCondicionalOLambda> | ϵ

<Return> ::= return <ExpresionOpcional>

<ExpresionOpcional> ::= <Expresion> | ϵ

<If> ::= if (<Expresion>) <Sentencia> <ElseOpcional>

<ElseOpcional> ::= else <Sentencia> | ϵ

<While> ::= while (<Expresion>) <Sentencia>

Una sola sentencia/expresión por separador. El `:` distingue el for de iteradores del estándar, y aparece justo después del declarador, así que el desempate es de un solo token.

<For> ::= for (<EncabezadoFor>) <Sentencia>

<EncabezadoFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional>

<EncabezadoFor> ::= var idMetVar <EncabezadoForTrasVar>

<EncabezadoFor> ::= <TipoPrimitivo> <DimensionesOpcionales> idMetVar <EncabezadoForTrasDeclarador>

<EncabezadoFor> ::= idGen <DimensionesOpcionales> idMetVar <EncabezadoForTrasDeclarador>

<EncabezadoFor> ::= idClase <EncabezadoForDesdeIdClase>

<EncabezadoFor> ::= <ExpresionSinIdClase> <ClausulasFinalesFor>

<EncabezadoForDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales> idMetVar <EncabezadoForTrasDeclarador>

<EncabezadoForDesdeIdClase> ::= . idMetVar <ArgsActuales> <ColaDeExpresionDesdePrimario> <ClausulasFinalesFor>

El `:` distingue el for de iteradores del estándar, y está a un solo token.

<EncabezadoForTrasVar> ::= : <Expresion>

<EncabezadoForTrasVar> ::= = <ExpresionCondicionalOLambda> <ClausulasFinalesFor>

<EncabezadoForTrasDeclarador> ::= : <Expresion>

<EncabezadoForTrasDeclarador> ::= <InicializacionDeDeclaradorFor> <ClausulasFinalesFor>

<InicializacionDeDeclaradorFor> ::= = <ExpresionCondicionalOLambda> | ϵ

Una sola expresión por separador.

<ClausulasFinalesFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional>

Las expresiones se organizan en capas, de menor a mayor precedencia: <Expresion> es la asignación (no asociativa, como en la gramática original); <ExpresionCondicional> el ternario; <ExpresionCompuesta> los operadores binarios, en un solo nivel como en el original; <ExpresionBasica> los operadores unarios prefijos; y <Operando> un literal, o una referencia con posfijo opcional. La lambda NO es una capa de esta jerarquía ni un <Primario>: es una alternativa del nivel MÁS ALTO, igual que en el JLS, que define `Expression: LambdaExpression | AssignmentExpression`. Por eso `->` no tiene precedencia contra ningún operador —no compite con ellos, vive arriba de todos—, y `1 + x -> x`, `- x -> x` y `a == x -> x` son inválidos, como en Java. Las dos cabezas que una lambda comparte con una expresión normal, `idMetVar` y `(`, se factorizan en <ExpresionDesdePrimarioOLambda>. Todo lado derecho de un `=` —el de la asignación y el de los cinco inicializadores— es una <ExpresionCondicionalOLambda> y no una <Expresion>. Eso preserva lo que la regla original conseguía usando <ExpresionCompuesta>: que `a = b = c` no sea válido.

<Expresion> ::= <ExpresionCondicionalOLambda> <RestoExpresion>

La <Expresion> de la gramática original: la que NO admite una lambda en la raíz. Se usa en los dos lugares donde Java tampoco la admite: adentro de una expresión parentizada que después va a ser operando, y como sentencia- expresión (`x -> x;` es «not a statement» en Java).

<ExpresionSinLambda> ::= <ExpresionCondicional> <RestoExpresion>

El lado derecho es ExpresionCondicionalOLambda y NO Expresion: la gramática original pone <ExpresionCompuesta> ahí, o sea que `a = b = c` NO es válido. Admite lambda porque el JLS define `Assignment: LeftHandSide = Expression`.

<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCondicionalOLambda> |
    ϵ

<OperadorAsignacion> ::= =

El nivel del JLS `LambdaExpression | ConditionalExpression`: admite lambda, no admite `=` en la raíz. Es el operando de los seis `=` y el tercer operando del ternario. Las cabezas que NO pueden empezar una lambda delegan en la jerarquía de siempre; las dos que sí —`idMetVar` y `(`— se factorizan. La cola se comparte con esa jerarquía, así que no hay lógica duplicada: sólo las cabezas.

<ExpresionCondicionalOLambda> ::= <OperadorUnario> <Operando>
    <ColaTrasOperandoSinAsignacion>

<ExpresionCondicionalOLambda> ::= <Primitivo>
    <ColaTrasOperandoSinAsignacion>

<ExpresionCondicionalOLambda> ::= <ExpresionDesdePrimarioOLambda>

<ExpresionDesdePrimarioOLambda> ::= idClase . idMetVar <ArgsActuales>
    <ColaDesdePrimarioSinAsignacion>

<ExpresionDesdePrimarioOLambda> ::= this <ColaDesdePrimarioSinAsignacion>

<ExpresionDesdePrimarioOLambda> ::= stringLiteral
    <ColaDesdePrimarioSinAsignacion>

<ExpresionDesdePrimarioOLambda> ::= <Creacion>
    <ColaDesdePrimarioSinAsignacion>

<ExpresionDesdePrimarioOLambda> ::= idMetVar <AccesoLlamadaOLambda>

<ExpresionDesdePrimarioOLambda> ::= <ParentizadaOLambda>

<ExpresionCondicional> ::= <ExpresionCompuesta> <RestoCondicional>

Asociativo a derecha, como en Java. El operando del MEDIO es una `<Expresion>` y el de la derecha una `<ExpresionCondicionalOLambda>`, tal cual el JLS 15.25: en Java `a ? b = 1 : c` es válido, `a ? b : c = d` no lo es, y `a ? b : x -> x` sí —la producción del JLS es `? Expression : LambdaExpression`—. No rompe LL(1) porque Primeros(Expresion) no contiene `:`.

`<RestoCondicional>` ::= **?** `<Expresion>` **:** `<ExpresionCondicionalOLambda>`

`<RestoCondicional>` ::= ϵ

`<ExpresionCompuesta>` ::= `<ExpresionBasica>` `<RestoExpresionCompuesta>`

`<RestoExpresionCompuesta>` ::= `<OperadorBinario>` `<ExpresionBasica>` `<RestoExpresionCompuesta>` | ϵ

`<OperadorBinario>` ::= **||** | **&&** | **==** | **!=** | **<** | **>** | **<=** | **>=** | **+** | **-** | ***** | **/** | **%**

`<ExpresionBasica>` ::= `<OperadorUnario>` `<Operando>`

`<ExpresionBasica>` ::= `<Operando>`

`<OperadorUnario>` ::= **+** | **-** | **!**

`<Operando>` ::= `<Primitivo>`

`<Operando>` ::= `<Referencia>` `<PosfijoOpcional>`

Va sobre `<Referencia>` y no sobre `<Operando>`: `5++` no tiene sentido.

`<PosfijoOpcional>` ::= **++**

`<PosfijoOpcional>` ::= **--**

`<PosfijoOpcional>` ::= ϵ

`<Primitivo>` ::= **true** | **false** | **intLiteral** | **charLiteral** | **null**

`<Primitivo>` ::= **floatLiteral**

Expresión que no puede empezar con idClase. Se usa donde una expresión compite con una declaración de variable local (sentencia y encabezado del for). Sólo duplica las *cabezas*: todas las colas se comparten con la jerarquía normal, así que no hay lógica repetida. NO admite lambda en la raíz, por la misma razón que `<ExpresionSinLambda>`: los dos contextos donde se usa son sentencia-expresión e inicialización del for, y en Java los dos exigen un StatementExpression, que nunca es una lambda.

`<ExpresionSinIdClase>` ::= `<OperadorUnario>` `<Operando>` `<ColaTrasOperando>`

`<ExpresionSinIdClase>` ::= `<Primitivo>` `<ColaTrasOperando>`

`<ExpresionSinIdClase>` ::= `<PrimarioSinIdClase>` `<ColaDeExpresionDesdePrimario>`

La cola se parte en dos para que el nivel `<ExpresionCondicionalOLambda>` pueda reusarla sin arrastrar el `=`, que ahí no corresponde.

`<ColaTrasOperando>` ::= `<ColaTrasOperandoSinAsignacion>` `<RestoExpresion>`

`<ColaTrasOperandoSinAsignacion>` ::= `<RestoExpresionCompuesta>` `<RestoCondicional>`

`<ColaDeExpresionDesdePrimario>` ::= `<RestoReferencia>` `<PosfijoOpcional>` `<ColaTrasOperando>`

`<ColaDesdePrimarioSinAsignacion>` ::= `<RestoReferencia>` `<PosfijoOpcional>` `<ColaTrasOperandoSinAsignacion>`

Ningún `<Primario>` puede ser una lambda: ésa es exactamente la corrección de precedencia. Un `<Primario>` es un operando, y una lambda no puede serlo.

<Referencia> ::= <Primario> <RestoReferencia>

<RestoReferencia> ::= <Encadenado> <RestoReferencia>

<RestoReferencia> ::= <AccesoArreglo> <RestoReferencia>

<RestoReferencia> ::= ϵ

<Encadenado> ::= . idMetVar <EncadenadoAtributoOMetodo>

Dedicado a este contexto: vacío = era <VarEncadenada>, con argumentos = era <MetodoEncadenado>.

<EncadenadoAtributoOMetodo> ::= <ArgsActuales> | ϵ

<AccesoArreglo> ::= [<Expresion>]

<Primario> ::= <LlamadaMetodoEstatico>

<Primario> ::= <PrimarioSinIdClase>

<LlamadaMetodoEstatico> ::= idClase . idMetVar <ArgsActuales>

<PrimarioSinIdClase> ::= this

<PrimarioSinIdClase> ::= stringLiteral

<PrimarioSinIdClase> ::= <Parentizada>

<PrimarioSinIdClase> ::= <AccesoVarOLlamada>

<PrimarioSinIdClase> ::= <Creacion>

El contenido es una <ExpresionSinLambda>: `(x -> x)` como OPERANDO no existe en Java —`(x -> x) + 1` y `(x -> x).m()` son «lambda expression not expected here»—. La lambda parentizada sí es válida cuando ella misma es toda la expresión, y ese caso lo cubre <ParentizadaOLambda>.

<Parentizada> ::= (<ExpresionSinLambda>)

Dedicado a este contexto: vacío = era <AccesoVar>, con argumentos = era <LlamadaMetodo>.

<AccesoVarOLlamada> ::= idMetVar <AccesoLlamada>

<AccesoLlamada> ::= <ArgsActuales> | ϵ

Las dos cabezas ambiguas, resueltas en el nivel más alto de la jerarquía. `(` abre tanto una expresión parentizada como la lista de parámetros de una lambda. Parece necesitar lookahead ilimitado (`(a,b,c) ->` vs `(a+b)`), pero NO lo necesita: - `( )` sólo puede ser una lambda de cero parámetros; - `( e , ...` sólo puede ser una lambda: MiniJava no tiene operador coma; - `( e )` se decide con el token que sigue al `)`: `->` es lambda, cualquier otro es una expresión parentizada. Con eso alcanza un token de lookahead en cada punto. Con flecha = lambda de un parámetro sin paréntesis; si no, es el <AccesoVar> o la <LlamadaMetodo> de siempre, con la cola de expresión que le corresponde.

<AccesoLlamadaOLambda> ::= -> <CuerpoLambda>

<AccesoLlamadaOLambda> ::= <AccesoLlamada>
<ColaDesdePrimarioSinAsignacion>

<ParentizadaOLambda> ::= (<ContenidoDeParentesis>

<ContenidoDeParentesis> ::=) -> <CuerpoLambda>

<ContenidoDeParentesis> ::= <Expresion> <RestoDeParentesis>

<RestoDeParentesis> ::=) <FinDeParentizadaOLambda>

<RestoDeParentesis> ::= , <ListaParametrosLambda>) -> <CuerpoLambda>

<FinDeParentizadaOLambda> ::= -> <CuerpoLambda>

<FinDeParentizadaOLambda> ::= <ColaDesdePrimarioSinAsignacion>

<ListaParametrosLambda> ::= idMetVar <RestoListaParametrosLambda>

<RestoListaParametrosLambda> ::= , idMetVar <RestoListaParametrosLambda> | ϵ

Una única expresión, sin llaves.

<CuerpoLambda> ::= <Expresion>

<Creacion> ::= new <ArregloOInstancia>

<ArregloOInstancia> ::= <TipoPrimitivo> <CreacionDeArreglo>

<ArregloOInstancia> ::= idGen <CreacionDeArreglo>

<ArregloOInstancia> ::= idClase <CreacionDesdeIdClase>

<CreacionDesdeIdClase> ::= < <GenericoDeCreacion>

<CreacionDesdeIdClase> ::= <CreacionDeArreglo>

<CreacionDesdeIdClase> ::= <ArgsActuales>

el diamante <> sólo se admite acá, en la rama que va a ArgsActuales: `new C<>()` es válido y `new C<>[3]` no. La restricción queda impuesta por la gramática, no delegada al semántico.

<GenericoDeCreacion> ::= > <ArgsActuales>

<GenericoDeCreacion> ::= <ArgumentoDeTipo> > <ArregloOInstanciaDeClase>

<ArregloOInstanciaDeClase> ::= <CreacionDeArreglo>

<ArregloOInstanciaDeClase> ::= <ArgsActuales>

Toda dimension de una creacion conserva su `[ Expresion ]`, que es la forma de MiniJava base: `CrearArreglo -> new TipoBase DimensionesConTamanio` con `DimensionesConTamanio -> [ Expresion ] DimensionesConTamanio | [ Expresion ]`. El logro de inicializacion NO afloja esa exigencia: le cuelga el inicializador al final. Asi `new int[2][2]{{1,2},{3,4}}` es valido y `new int[]{1,2,3}` no. Esto DIVERGE de Java, que prohibe tamanio e inicializador juntos (JLS 15.10.1, verificado con javac: «array creation with both dimension expression and initialization is illegal»). Es una decision deliberada: se prefirio no ampliar la forma de las dimensiones que MiniJava ya tenia definida. Revertirla es cambiar CreacionDeArreglo y nada mas.

<CreacionDeArreglo> ::= <DimensionesConTamanio> <InicializadorDeArregloOpcional>

<DimensionesConTamanio> ::= [<Expresion>] <DimensionesConTamanioOpcional>

<DimensionesConTamanioOpcional> ::= <DimensionesConTamanio> | ϵ

El inicializador es anidable: un elemento puede ser una expresion o, a su vez, otro inicializador. Solo es alcanzable desde <Creacion>, que es lo unico que el logro restringe sobre donde puede aparecer.

<InicializadorDeArregloOpcional> ::= <InicializadorDeArreglo> | ϵ

<InicializadorDeArreglo> ::= { <ListaElementosOpcional> }

<ListaElementosOpcional> ::= <ListaElementos> | ϵ

<ListaElementos> ::= <ElementoInicializador> <RestoListaElementos>

<RestoListaElementos> ::= , <ElementoInicializador> <RestoListaElementos> | ϵ

<ElementoInicializador> ::= <Expresion> | <InicializadorDeArreglo>

Sección propia y no parte de la creación: la lista de argumentos de una llamada la usan por igual el `new`, el encadenado, la llamada estática y el encabezado del for.

<ArgsActuales> ::= (<ListaExpsOpcional>)

`<ListaExpsOpcional>` `::=` `<ListaExps>` | ϵ

`<ListaExps>` `::=` `<Expresion>` `<RestoListaExps>`

`<RestoListaExps>` `::=` `,` `<Expresion>` `<RestoListaExps>` | ϵ

8 Ambigüedades resueltas

Las tres ambigüedades del registro de la sección 11 se resuelven todas por regla de implementación —la vía que autoriza Clase 4, slide 49— y todas de la misma forma: **voraz**, consumiendo todo lo que se pueda:

Caso	Decisión
dangling else	El **else** liga al **if** más cercano. En el analizador es el orden de las ramas: se toma **else** apenas se lo ve. Coincide con Java.
`new int[3][4]`	Todas las dimensiones pertenecen a la creación; no es un acceso al arreglo recién creado. Coincide con Java, que lo lee como una creación 2-D. Heredada de la gramática de la cátedra.
Cuerpo de la lambda	`x -> a = b` es `x -> (a = b)`. La introduce este trabajo, y afecta a un solo no terminal: `<RestoExpresion>`, la única cola externa que todavía compite con la del cuerpo una vez que la lambda vive en el nivel más alto de la jerarquía (sección 4.3). Coincide con Java.

Además, `<Asignacion>` y `<Llamada>` tienen cuerpos idénticos en la gramática original y por lo tanto toda sentencia-expresión tiene dos árboles. Se **colapsan en una sola producción**: distinguir una asignación de una llamada exige saber a qué se refiere el nombre de la izquierda, o sea una tabla de símbolos, y eso es trabajo del analizador semántico.

9 El analizador sintáctico

9.1 La técnica

Analizador **Descendente Recursivo Predictivo LL(1)**, con el **Esquema Simple** de Clase 5: un método por no terminal, `match` para los terminales, y la producción se elige mirando si el token actual está en los Primeros del cuerpo de alguna de ellas. **Sólo Primeros**: los Siguientes no aparecen en ninguna línea del compilador.

Los métodos tienen tres formas, según cuántas producciones tenga el no terminal y si alguna deriva a vacío:

Forma del NT	Traducción
Una sola producción	El cuerpo, sin condición.
Varias, ninguna anulable	Una cadena de `if/else if` sobre los Primeros de cada cuerpo, y un `else` que lanza el error (slide 36).
Varias, alguna anulable	Lo mismo, pero el `else` **no hace nada**: que el token no coincida con ninguno significa que el no terminal derivó a vacío, y quien llamó decidirá si eso es un error (slides 52–54).

Ese `else` vacío **se escribe igual**, con la producción ε citada al lado. El analizador se comporta igual si se lo omite, pero omitirlo borra del código el único lugar donde está dicho «acá el no terminal derivó a vacío» —que es donde la Etapa 3 va a colgar la traducción de esa producción—.

La **recursión a derecha se traduce con recursión**, como en la slide 59, y no con un bucle. El analizador no se toma **ninguna** libertad respecto del esquema.

9.2 Las clases

El analizador tiene 111 no terminales. Escribirlos en una sola clase daba 1176 líneas y 89 métodos, así que están repartidos en **una clase por sección de la gramática**: el archivo de código y la sección del `.g` se leen uno al lado del otro.

Clase	Qué traduce
`AnalizadorSintactico`	El arranque y el contrato de uso: carga el primer token, llama al símbolo inicial y devuelve la lista de errores.
`EntradaDeTokens`	El lookahead y **el único punto que consume entrada**.
`Sintaxis`	Cablea las once secciones. Existe porque la gramática tiene recursión mutua entre ellas —una sentencia contiene expresiones, una expresión contiene lambdas, el cuerpo de una lambda vuelve a ser una expresión— y por lo tanto no se pueden inyectar unas en otras por constructor sin un ciclo.
`ParteDelAnalizador`	La base de las diez: delega `match`, `actual` y `estaEn` para que cada método se lea como en la slide 57.
`AnalizadorDe{Clases, Miembros, Tipos, Sentencias, Fors, Expresiones, Referencias, Lambdas, Creacion, Argumentos}`	Una sección de la gramática cada una, entre 99 y 173 líneas.
`primeros/PrimerosDeX`	Una constante por producción, **generada** desde la gramática: 230 en total, divididas por la misma sección.
`recuperacion/ {RecuperadorDePanico, ContextoDeSincronizacion}`	La recuperación en Modo Pánico.
`errores/ExcepcionSintactica`	El único error que produce el analizador.

Que el código y la gramática no se separen no depende de la disciplina: cada método cita arriba las producciones que traduce, y `auditar_analizador.py` falla si una producción no la implementa nadie o si un comentario quedó viejo (sección 11).

9.3 Los tipos de error que detecta

Todo error que emite el analizador informa **las dos mitades**: qué se encontró —el lexema real, entre comillas— y qué se esperaba en su lugar. No es una elección de redacción: son literalmente los dos datos con los que el Esquema Simple decide, el token actual y el conjunto de Primeros de la producción, así que el mensaje es la traducción directa de la condición que falló.

El Esquema Simple tiene **exactamente dos** puntos donde puede fallar, y no hay un tercero:

1. **Un `match` que no coincide.** Se esperaba un terminal concreto y llegó otro. *«se esperaba `;` se encontro `"void"`».*

2. **Un no terminal NO anulable cuyo token actual no está en los Primeros de ninguna de sus producciones.** *«se esperaba `this`, `new`, ... o `(` se encontro `";"`».* Cuando el no terminal *sí* es anulable esto no es un error: derivó a vacío y lo resolverá quien llamó.

En el primer caso «lo que se esperaba» es un terminal único; en el segundo, la lista completa de los terminales que podrían haber seguido, que es exactamente el conjunto de Primeros consultado.

A esos se suman los **cuatro chequeos del logro MiniLambdas** que una gramática libre de contexto no puede expresar (sección 12): el `this` explícito dentro de una lambda, la captura de una variable que no es parámetro, que un parámetro sea un identificador sin tipo y no se repita, y que una lambda parentizada no lleve cola de expresión. Rechazan con el mismo código `[Error:lexema|línea]` y, como ahí «lo esperado» no es un conjunto de tokens sino una regla, declaran el motivo en prosa —*«no se permite this dentro de una lambda»*, *«una lambda no puede ser operando de .»*— sin perder la mitad de «qué se encontró».

Y una aclaración que conviene dejar escrita: los errores **léxicos** los sigue detectando y reportando el analizador de la Etapa 1, con su propio formato. El sintáctico no los toca.

9.4 El formato del reporte

Pautas slide 25: un mensaje que nombre el lexema y la línea, un renglón en blanco y el código. El mensaje abre con lo esperado y cierra con lo encontrado, en ese orden.

```
Error Sintactico en linea 3: se esperaba idClase se encontro "clasesita"

[Error:clasesita|3]
```

El lexema del token EOF es `$`, como fija Pautas slide 26. Con el logro *Modo Pánico* prendido el reporte entero se repite por cada error encontrado, en orden, que es el mismo formato con el que la Etapa 1 reporta varios errores léxicos.

10 Cómo compilar y correr

El proyecto se construye con Maven y apunta a Java 25. Desde la carpeta `compilador/`, que es donde está el `pom.xml`:

```
$ mvn package          # compila, corre la suite y arma target/Compilador.jar
$ mvn verify           # ademas verifica los umbrales de cobertura
$ mvn test             # solo la suite
```

El jar resultante es ejecutable y recibe el fuente como único argumento, tal cual pide Pautas slide 28. Acepta cualquier extensión:

```
$ java -jar target/Compilador.jar programa1.java
Compilacion Exitosa

[SinErrores]
```

Hay una sola opción más, que no hace falta para la corrección de esta etapa:

```
# La Etapa 1 sigue siendo ejecutable sola: vuelca la lista de tokens.
$ java -jar target/Compilador.jar --etapa=1 programa1.java
```

No hay forma de elegir con qué logros correr: el compilador reconoce MiniJava con todos los de la Etapa 1 y la Etapa 2 incorporados, siempre.

Una opción desconocida se informa por salida estándar y **no** aborta: los testers de la cátedra invocan `main()` dentro del mismo proceso, y una excepción o un `System.exit()` matarían la corrida entera. Por el mismo motivo el compilador no llama nunca a `System.exit()`.

11 Verificación

Los conjuntos de Primeros se calcularon a mano y se contrastaron con `herramientas/primeros.py`, escrita para este trabajo. La herramienta **contrasta** la cuenta manual, no la reemplaza: cuando las dos difirieron, primero se resolvió cuál estaba equivocada.

11.1 Condición LL(1)

```
$ python3 herramientas/primeros.py gramatica/minijava-base.g
--- 60 no terminales, 126 producciones, 51 terminales ---
NO ES LL(1): 16 conflicto(s)

$ python3 herramientas/primeros.py gramatica/minijava-ll1-final.g
--- 112 no terminales, 230 producciones, 60 terminales ---
ES LL(1): toda produccion de todo no terminal tiene Primeros disjuntos.
```

11.2 Ambigüedad frente a conflicto de predicción

Para distinguir los dos defectos de la sección 3.2 se comparan los rangos de longitud derivable: si no se solapan, los lenguajes son disjuntos y no puede haber ambigüedad.

```
$ python3 herramientas/primeros.py gramatica/minijava-base.g \
      --longitudes VarEncadenada MetodoEncadenado
<VarEncadenada>: entre 2 y 2 tokens
<MetodoEncadenado>: entre 4 y inf tokens

Los rangos NO se solapan: ninguna cadena puede estar en los dos
lenguajes, asi que <VarEncadenada> y <MetodoEncadenado> NO son ambiguos
entre si. Si comparten prefijo izquierdo es un conflicto de PREDICCION
(LL(1)), no de ambiguedad, y se arregla factorizando.
```

11.3 Registro de ambigüedades

La condición LL(1) que verifica el apartado anterior es la de la cátedra: disjunción de Primeros. El ítem «no es ambigua, o es arreglable» del checklist de Clase 4, slide 41, es otro, y se audita aparte. La herramienta junta toda ambigüedad que aparezca y **exige que cada una esté declarada con su regla de resolución**: si aparece una nueva, falla.

```
$ python3 herramientas/primeros.py gramatica/minijava-ll1-final.g --ambiguedades
DIAGNOSTICO OFFLINE - no forma parte del metodo de implementacion.
El analizador sintactico decide SOLO con Primeros (Esquema Simple,
```

```
Clase 5). Los Siguientes se usan aca, y solo aca, para auditar el item
"no es ambigua o es arreglable" del checklist de Clase 4 slide 41.

Ambiguedades detectadas: 3

  <DimensionesConTamanioOpcional>  ...  [declarada]
  <ElseOpcional>                   ...  [declarada]
  <RestoExpresion>                 ...  [declarada]

Todas las ambiguedades detectadas estan declaradas con su regla de resolucion.
```

Eso permite afirmar, con evidencia y no por revisión a ojo, que **éstas son todas las ambigüedades de la gramática** y que cada una tiene una resolución explícita.

11.4 Casos de aceptación y rechazo

La herramienta incluye un simulador del esquema de traducción — elige la producción cuyo cuerpo tiene al token actual en sus Primeros, y si ninguna lo tiene pero el no terminal es vaciable, no hace nada — que permite probar la gramática **antes** de escribir el analizador. Se verifican **81 casos**: por cada construcción, la sintaxis que debe aceptar y la que debe rechazar.

```
$ bash herramientas/casos-gramatica.sh
--- nucleo ---
  ok    clase vacia
  ok    if/else anidado
  ok    encadenamiento
--- cada construccion contra su propia sintaxis ---
  ok    x -> x            (1 param sin parentesis)
  ok    ( ) -> 1          (0 params)
  ok    ( a ) -> a        (1 param con parentesis)
  ok    ( a , b ) -> a    (2 params)
  ok    ( a ) + 1         (parentizada, NO lambda)
  ok    cuerpo con llaves NO se permite
  ok    parametro con tipo NO se permite
  ok    List < List < Persona > > x ;  (anidado)
  ok    new List < > ( )               (diamante)
  ok    List < > x ;    diamante en posicion de TIPO
  ok    new List < > [ 3 ]  diamante con arreglo
--- orden de modificadores: los dos, como en Java ---
  ok    public static void f(){}
  ok    static public void f(){}
  ok    public private void f(){}
  ok    static static void f(){}
...
TODOS LOS CASOS PASAN.
```

Los casos que fijan las decisiones difíciles de este informe son, entre otros: `a = b = c` se rechaza (no asociatividad de la asignación); `( a ) + 1` se acepta como expresión parentizada y `( a ) -> a` como lambda, decidiéndose con un solo token; `List<> x;` se rechaza y `new List<>()` se acepta; `static public` y `public static` se aceptan los dos y `public private` no; `new int[2]{1,2}` se acepta y `new int[]{1,2}` no.

11.5 Validación contra los casos de la cátedra

La verificación más fuerte no usa casos propios. La herramienta `aceptar_fuente.py` tokeniza un fuente **real** con el analizador léxico de la Etapa 1 y le pasa la secuencia de tokens al simulador, de modo que la gramática se puede probar contra los ocho casos entregados con los testers de esta etapa antes de que exista el analizador sintáctico.

```
$ python3 herramientas/aceptar_fuente.py \
      ../vendor/testersSintactico/resources/*/*.java
  ok    sintCorrecto01.java      acepta
  ok    sintCorrecto02.java      acepta
  ok    sintCorrecto03.java      acepta
  ok    sintCorrecto04.java      acepta
  ok    sintError01.java         rechaza  [Error:,|3] ==
        se esperaba ‘{‘ y se encontro ‘,‘
  ok    sintError02.java         rechaza  [Error:clasesita|3] ==
        se esperaba ‘idClase‘ y se encontro ‘idMetVar‘
  ok    sintError03.java         rechaza  [Error:hola|5] ==
        se esperaba ‘;‘ y se encontro ‘idMetVar‘
  ok    sintError04.java         rechaza  [Error:else|7] ==
        se esperaba ‘}‘ y se encontro ‘else‘

TODOS LOS CASOS PASAN.
```

Los cuatro casos válidos se aceptan y los cuatro inválidos se rechazan. Además —y esto es lo que importa para la corrección— el rechazo se produce **en el token exacto** que declara la expectativa de cada caso: el `==` indica que el código `[Error:lexema|línea]` derivado de la gramática coincide con el que la cátedra espera en los cuatro. La gramática no sólo reconoce el lenguaje correcto: falla donde tiene que fallar.

11.6 Tres reconocedores independientes de la misma gramática

El argumento de corrección más fuerte no es que el analizador acepte una lista de casos, sino que **tres implementaciones escritas por separado coincidan**. Las tres salen del mismo archivo de gramática y no comparten una línea de código:

El analizador de la entrega Java, descendente recursivo, Esquema Simple de Clase 5. Elige mirando los Primeros que `primeros_a_java.py` genera desde la gramática.

El simulador Python, con pila explícita, en `primeros.py`. Imita el mismo esquema pero sin recursión, y existía antes que el analizador.

El oráculo JavaCC generado por `gramatica_a_javacc.py`. JavaCC es un generador de parsers de terceros: se le da la gramática y escribe el parser. Con `LOOKAHEAD = 1` usa exactamente la misma técnica.

Decisión — qué prueba, y qué no prueba, que JavaCC no emita avisos

JavaCC avisa `Choice conflict involving two expansions` cuando dos alternativas de un mismo no terminal empiezan con el mismo token. Sobre la gramática final **no emite ninguno**, lo que es una confirmación independiente —de una herramienta que calculó los conjuntos por su cuenta sobre las 230 producciones— de la condición LL(1) de la cátedra: disjunción de Primeros.

No confirma la condición fuerte. Verificado con dos gramáticas de control: ante `<S>` → **a b** | **a c** avisa; ante `<Opt>` → **b** | ε seguido de **b** —un conflicto Primeros–Siguientes— no avisa nada. Las tres ambigüedades declaradas son de esa segunda clase, así que su silencio no dice nada sobre ellas.

Lo que sí lo hace útil como oráculo es cómo resuelve lo que no puede predecir: toma la primera alternativa que coincide, que es la misma resolución voraz que aplica este analizador. Por eso las tres coinciden también en los casos ambiguos, y por eso el oráculo puede etiquetar el corpus.

11.7 Equivalencia de lenguajes con la gramática de la cátedra

Clase 4 exige adaptar la gramática *«sin cambiar el lenguaje que genera»*. Eliminar recursión a izquierda y factorizar lo preservan *en teoría*; `equivalencia_de_lenguajes.py` lo comprueba *en la práctica* sobre las dos gramáticas concretas. La comparación es contra el **núcleo LL(1) sin logros**, `minijava-ll1.g`: es lo que tiene que generar el mismo lenguaje que la cátedra, porque los logros por definición lo agrandan. Ese archivo no lo consume el compilador; existe exactamente para esto.

La herramienta deriva una cadena por cada producción de cada gramática —dos por cada producción anidable, porque un solo uso no distingue `a = b` de `a = b = c`— y verifica que la otra la acepte. Como la gramática base no es LL(1), reconocerla necesita un algoritmo que tolere recursión a izquierda y ambigüedad: se usa un **reconocedor de Earley**.

```
$ python3 herramientas/equivalencia_de_lenguajes.py
minijava-base.g: 126/126 producciones ejercitadas, 0 inertes
minijava-ll1.g:  135/135 producciones ejercitadas, 0 inertes

551 mutantes comparados en las dos gramaticas.
LAS DOS GRAMATICAS GENERAN EL MISMO LENGUAJE: toda produccion de cada una fue
ejercitada y la otra estuvo de acuerdo.
```

La cobertura por producción es lo que le da fuerza al chequeo: se validó contra dos controles negativos —uno que ensancha el lenguaje permitiendo `a = b = c` y otro que lo angosta quitando `<Sentencia>` → `<Bloque>`— y los detecta a los dos.

11.8 Contraste con un corpus ajeno

Todas las verificaciones anteriores comparten un límite: las implementaciones que se contrastan salen del *mismo* archivo de gramática, así que comparten sus decisiones. El contraste que cierra ese hueco usa un corpus de **30.000 fuentes generados a partir de otra gramática** de MiniJava, que no comparte con este trabajo ni el código ni los criterios.

Decisión — el corpus y su generador son de Giuliano Taormina

El *fuzzer* que produjo esos 30.000 casos, y la gramática de la que se derivan, los escribió **Giuliano Taormina**, compañero de la materia, que los compartió para que cada uno pudiera contrastar su analizador. No son material de este trabajo: acá sólo se corrió el compilador contra ellos y se clasificó cada divergencia. El crédito de la herramienta es suyo.

26.291 coinciden exactamente. Los 3.709 restantes caen en cinco familias, todas declaradas y verificadas a mano:

Familia	Casos	Qué la explica
`(x -> y).m()`, `(x -> y) + 1`, `(x -> y)[0]`	2434	**Dos lecturas del enunciado del logro**, y las dos defendibles. Ver la discusión de abajo.
`"a\\"`	987	**Este trabajo**, por lectura de la especificación: *Aspectos Léxicos* §2.3.3 da una regla **posicional** —la comilla de cierre nunca está antecedida por \— y no una de apareamiento. Su generador asume la semántica de Java.
`new Clase<Arg>()`	162	**Este trabajo.** La gramática de la cátedra lo genera (`<LlamadaConstructor>` `::=` **new** `<TipoReferencia>` `<ArgsActuales>`, con `<TipoReferencia>` `::=` **idClase** `<TipoGenericoOpcional>`) y la suya lo rechaza.
`public void m(){}`, constructor sin **public**	6	**Ninguno**: sus corpus se generaron sin el logro de *Visibilidad Mejorada*.
Los dos rechazan, en distinto token	120	**Ninguno**: su gramática cubre menos construcciones, así que el primer token que no encaja es otro.

Decisión — la lambda parentizada con cola: dos lecturas del mismo enunciado

La familia más grande no es un defecto de ninguno de los dos analizadores: es que el enunciado del logro admite dos lecturas, y cada uno tomó una.

La lectura de la otra implementación. MiniJava base ya define que *cualquier* expresión parentizada es un `<Primario>`, y que un `<Primario>` arrastra cadena:

```
<Primario> ::= <ExpresionParentizada>
<ExpresionParentizada> ::= ( <Expresion> )
<Referencia> ::= <Referencia> <MetodoEncadenado>
```

Si el logro agrega la lambda como alternativa de `<Expresion>` —que es el único no terminal de expresión que la gramática tiene—, entonces `(x -> y).m()` **se deriva solo**, sin tocar ninguna regla. Restringirlo obliga a achicar `<ExpresionParentizada>`, o sea a modificar una regla que la especificación ya daba.

La lectura de este trabajo. El logro dice *«con una sintaxis similar a la utilizada en Java»*, y en Java la lambda no es un *Primary*: el JLS la define como alternativa de *Expression*, el nivel más alto. `javac` rechaza los tres casos con *«lambda expression not expected here»*. Como MiniJava base no tiene lambdas, dónde va en la jerarquía es un hueco, y la regla de la cátedra para los huecos es «como en Java». Acá la lambda se produce en un nivel **nuevo, por encima** del `<Expresion>` original, así que `<ExpresionParentizada>` sigue conteniendo exactamente lo que contenía —lo que cambia es que hay un nivel más arriba que ella no alcanza—.

Cuál pesa más depende de si se considera que la ubicación de la lambda es un hueco de la especificación (y entonces manda Java) o una consecuencia mecánica de reglas que la especificación ya fijó (y entonces manda la base). Este trabajo tomó la primera; queda declarado que la segunda es defendible.

Las cinco están declaradas en el test, que falla si aparece una sexta. Es la misma disciplina que el registro de ambigüedades: la divergencia conocida se documenta, la desconocida rompe la suite.

11.9 El analizador cita exactamente la gramática

Cada método del analizador lleva arriba, en un comentario, las producciones que traduce. `auditar_analizador.py` compara esas citas contra la gramática y falla si una producción no la implementa nadie o si un comentario quedó viejo:

```
$ python3 herramientas/auditar_analizador.py
230 producciones en la gramatica, 230 citadas por el analizador.
EL ANALIZADOR CITA EXACTAMENTE LAS PRODUCCIONES DE LA GRAMATICA.
```

12 Decisiones tomadas con criterio propio

Buena parte del trabajo de esta etapa no fue aplicar transformaciones mecánicas sino *decidir* dónde la especificación admitía más de una lectura. Esta sección junta todas esas decisiones en un solo lugar, cada una con la regla que se aplicó y con la fuente que la respalda.

El criterio de desempate, cuando lo hubo, es el que fija la cátedra: *«todo lo que NO esté especificado para MiniJava tiene comportamiento por defecto igual al de Java»*.

12.1 Dónde la especificación admitía más de una lectura

Trece decisiones, cada una desarrollada en la sección que le corresponde. Esta tabla existe para poder recorrerlas de un vistazo, no para repetirlas.

Dónde	Qué se decidió	Ver
Asociatividad del ternario	A derecha: `a ? b : c ? d : e` es `a ? b : (c ? d : e)`, como en Java.	§6
Los dos operandos del ternario	El del medio es una `<Expresion>` completa y el de la derecha no, tal cual el JLS 15.25: `a ? b = 1 : c` es válido y `a ? b : c = d` no.	§6
La asignación no encadena	`a = b = c` nunca fue válido en MiniJava; se preservó. **No es una decisión libre**: está en la regla original.	§5
Genéricos anidados	`List<List<Persona>>` cierra con dos $>$ separados: el léxico no tiene un token $>>$, así que el problema clásico de Java no se presenta.	§6
Visibilidad en métodos de interfaz	Se admite **public** y **no private**, porque en Java un método de interfaz privado exige cuerpo y MiniJava no tiene forma de escribirlo.	§6.1
Orden de los modificadores	Los dos: `public static` y `static public`. Repetir el modificador se rechaza, y lo impone la gramática sola.	§6.1
Inicializador de arreglos	Toda dimensión conserva su tamaño y el inicializador se cuelga al final. **Diverge de Java**, deliberadamente.	§6
Notación diamante	Sólo al instanciar: `new C<>()` sí, `new C<>[3]` y `List<> x;` no. Impuesto por la gramática, no delegado al semántico.	§6
El declarador tipado del **for**	`for (int i = 0; ; )` funciona por sí mismo: el encabezado tiene su propio inicializador y no reusa el de las variables locales.	§6
float en la gramática	Se agrega como `<TipoPrimitivo>` y como `<Primitivo>`; sin eso el logro de la Etapa 1 quedaba inutilizable en ésta.	§6

Tres no aparecen en ninguna otra sección, así que van acá:

Dimensiones del arreglo. `int[] x` se acepta y `int x[]` **no**: las dimensiones van pegadas al tipo base y nunca al nombre, que es lo que define MiniJava base. Java admite las dos formas, pero acá la especificación *sí* dice algo y gana sobre la regla de desempate.

Herencia. Una clase hereda de **una** clase o implementa **una** interfaz, nunca las dos ni varias: es la regla de la cátedra tal cual (`<HerenciaOpcional>` ::= **extends** `<TipoReferencia>` | **implements** `<TipoReferencia>` | ϵ). Java admite `extends A implements B, C`; no se amplía.

El lexema del token EOF. Es `$`. El material de la cátedra se contradice: Pautas slide 26 y *Aspectos Léxicos* §2.5 dicen `$`, y la *Guía de uso de los testers — Etapa 2*, §5, dice que es vacío y que el código queda `[Error:|N]`. Como los dos formatos son incompatibles bajo la comparación por contenido que hace el tester, se consultó y se confirmó `$`.

12.2 Qué verifica el analizador y qué queda para el semántico

El criterio de frontera es uno solo:

> **Si la restricción se decide con información que el analizador acaba de leer y sigue teniendo a mano, la verifica el analizador sintáctico. Si hace falta resolver un nombre contra su declaración —o sea, una tabla de símbolos con alcances— es del analizador semántico.**

Lo que sigue es el inventario completo de lo que **atraviesa** el análisis sintáctico y queda para la Etapa 3. Está separado en dos: lo que viene de MiniJava base y lo que agrega cada logro de esta etapa.

Del núcleo de MiniJava.

Pasa el análisis sintáctico	Qué tiene que decidir el semántico
`int x = "hola";`	compatibilidad de tipos en la inicialización
`noDeclarada = 5;`	que el nombre esté declarado y en alcance
`int x; int x;`	declaraciones duplicadas en el mismo alcance
`5 = x;`	que el lado izquierdo de un `=` sea asignable
`m(1,2,3)` con `m` de un argumento	aridad contra la declaración
`x.metodoQueNoExiste()`	que el miembro exista en el tipo del receptor
`return;` en un método `int`	compatibilidad del valor de retorno
un método `int` sin `return`	que todo camino devuelva un valor
`class A extends UnaInterfaz {}`	que el padre de un `extends` sea una clase
`class A implements OtraClase {}`	que lo implementado sea una interfaz
`class A extends A {}`	ciclos en la jerarquía de herencia
`new Inexistente()`	que la clase exista y tenga ese constructor
`this` en un método `static`	que haya una instancia sobre la cual resolverlo
`a[i]` con `a` de tipo `int`	que el receptor de un `[` sea un arreglo
`if (1)`	que la condición sea `boolean`
`C.m()` con `m` de instancia	que el método invocado así sea `static`

De los logros de esta etapa.

Logro	**Pasa el análisis sintáctico**	**Qué tiene que decidir el semántico**
MiniLambdas	`x -> this.campo`	Que no haya **this** *explícito* dentro del cuerpo.
MiniLambdas	`x -> campo`	Que no haya **this** *implícito*: si `campo` es un atributo, lo hay. Misma forma que el caso de abajo, y por eso los dos van juntos.
MiniLambdas	`x -> x + n`	Que `n` no sea una variable local ni un parámetro de afuera, o sea una captura.
MiniLambdas	`x -> y -> x + y`	Ídem con lambdas anidadas: la interna usa el parámetro de la externa.
MiniLambdas	`x -> ayuda(x)`	Es un **this** implícito *sólo si* `ayuda` es un método de instancia; si es **static** de la misma clase, Java lo acepta.
MiniLambdas	`(a, a) -> a`	Que dos parámetros de la misma lambda no se llamen igual.
MiniLambdas	`(a ? b : x -> x).m()`	Un ternario con una rama lambda es una *poly expression* (JLS 15.25): lo decide la clasificación de la expresión y su tipo destino.
Genericidad	`Lista<Persona> l = new Lista<Auto>();`	Que el argumento de tipo cierre con el del tipo declarado.
Genericidad	`C<T> x;` con `C` no genérica	Que la clase instanciada declare un parámetro de tipo.
Arreglos	`new int[2]{1,2,3}`	Que la cantidad de elementos del inicializador coincida con el tamaño, y que cada elemento sea del tipo del arreglo.
Arreglos	`new int[2][2]{{1},{2,3}}`	Que el anidamiento del inicializador coincida con la cantidad de dimensiones.
Fors!	`for (var e : 5) ;`	Que lo recorrido por el **for** de iteradores sea iterable.
Visibilidad	`otro.metodoPrivado()`	Que un miembro **private** no se use desde afuera de su clase.
Posfijos	`5++` en una variable no numérica	Que el operando sea una variable de tipo numérico y asignable.
Atributos inic.	`int x = otroAtributo;`	Orden de inicialización y alcance de lo que se referencia.

Y lo que sí verifica el analizador, aunque la gramática no lo exprese. Son dos, los dos del logro de lambdas, y los dos son restricciones de *forma*: no hay ningún nombre que resolver, alcanza con lo que el analizador acaba de leer.

Se rechaza	Cómo se decide
`(a + 1) -> a`	El contenido del paréntesis se lee como `<Expresion>` —hasta ver el `->` no se sabe si era una lista de parámetros— y se recuerda si consumió exactamente un **idMetVar**. Del segundo parámetro en adelante lo impone la gramática.
`(x -> x).m()`	Se recuerda si la **raíz** de lo que se acaba de leer entre paréntesis fue una lambda, y si lo fue se exige que no siga ninguna cola. Expresarlo en la gramática exigiría clonar la cadena de colas entera en una variante «el contenido fue una lambda», unos siete no terminales.

12.3 Decisiones de implementación

Insertar el no terminal, no fusionar. Al factorizar, cuando había que elegir entre fundir dos contextos en un mismo no terminal o insertar uno nuevo por contexto, se insertó. Clase 4, slide 40, avisa que factorizar de más le complica el trabajo al semántico; el no terminal insertado es justamente el punto de decisión que la Etapa 3 va a necesitar.

La recursión a derecha se traduce con recursión. Clase 5, slide 59, muestra el método de `<E'>` llamándose a sí mismo, y así está implementado en los diez lugares donde la gramática tiene una producción `<X>` $\rightarrow \alpha$ `<X>` $\mid \epsilon$. Hubo una versión con `while` —deriva la misma cadena y no hace depender de la pila de la JVM una cadena larguísima de operadores—, y se revirtió por indicación del profesor: aleja el código del esquema que se extiende en el semántico y lo vuelve más propenso a errores. El analizador no se toma **ninguna** libertad respecto del Esquema Simple.

Modo Pánico: dónde se recupera. En tres lugares y sólo tres —las tres listas de la gramática que repiten un elemento: clases, miembros y sentencias—, así el manejo del error no se desparrama por las diez secciones del analizador. Son justamente las tres producciones recursivas a derecha del punto anterior, de modo que la recuperación vive dentro de la recursión: el método de la lista corre un elemento, y si falla registra el error, sincroniza y se vuelve a llamar sólo si se puede seguir. La invariante que lo hace terminar es que cada nivel consuma al menos un token.

Modo Pánico: con qué se sincroniza. Con los tokens de Pautas slide 40 (`;`, `}` y la llave que abre) **más los Primeros del elemento de la lista**. Los Primeros hacen falta: sin ellos, un `int x` al que le falta el `;` se lleva puesto el método que viene después, porque el análisis falla justo en el **void** con el que empieza el miembro siguiente y correcto. Las llaves se cuentan mientras se descarta, así el cuerpo de un método con la firma rota se saltea entero en vez de leerse como si cada sentencia fuera otro miembro.

Modo Pánico: el formato del reporte. Se repite el reporte completo de Pautas slide 25 —mensaje, renglón en blanco, código— por cada error. Es el mismo formato con el que la Etapa 1 reporta varios errores léxicos con *Multi-detección*.

Modo Pánico: el primer error no cambia. La recuperación agrega errores *después* del primero, nunca cambia cuál es el primero: el primer código reportado es exactamente el que daría un analizador que cortara ahí, que es la convención base de Pautas slide 25. Cada caso del corpus con errores declara ese primer código, y el tester estricto verifica que sea el primero que el compilador emite.

13 Herramientas desarrolladas

Ninguna forma parte del compilador: viven en `compilador/herramientas/` y existen para **verificar** lo que este informe afirma. La gramática se mantiene en texto plano en `compilador/gramatica/`, y los documentos derivados —el listado de producciones y la tabla de Primeros de este informe, y las clases `PrimerosDeX` del analizador— se **generan** desde ahí y no se editan a mano, para que no puedan divergir.

`primeros.py` El verificador. Calcula símbolos vaciables y conjuntos de Primeros por punto fijo, verifica la condición LL(1) de la cátedra, compara rangos de longitud derivable para distinguir una ambigüedad de un conflicto de predicción, audita el registro de ambigüedades declaradas, y trae un simulador del Esquema Simple que permite probar la gramática sobre una secuencia de tokens antes de escribir una línea de Java.

`primeros_a_java.py` Genera las diez clases `PrimerosDeX` que el analizador consulta para elegir producción. Es lo único que garantiza que lo que el analizador usa sea exactamente lo que el verificador calcula.

`gramatica_a_latex.py` Genera el listado de producciones y la tabla de Primeros de este informe, desde el mismo archivo que verifica `primeros.py`.

`gramatica_a_markdown.py` Ídem para la documentación técnica del repositorio.

`gramatica_a_javacc.py` y `gramatica_a_antlr.py` Traducen la gramática a los formatos de dos generadores de parsers de terceros. El parser JavaCC es el **oráculo** que etiqueta el corpus generado (sección 11.6); la gramática ANTLR alimenta al generador de casos.

`equivalencia_de_lenguajes.py` Un **reconocedor de Earley** escrito para este trabajo, que comprueba que corregir la gramática de la cátedra no cambió el lenguaje que genera, derivando una cadena por cada producción de cada una.

`auditar_analizador.py` Compara las producciones que cada método del analizador cita en su comentario contra la gramática, y falla si una producción no la implementa nadie o si un comentario quedó viejo.

`comparar_con_gramatica.py` Corre el analizador en Java y el simulador en Python sobre el corpus entero y exige el mismo veredicto *y el mismo token de rechazo*. Es la verificación más fuerte de la suite: no depende de que a alguien se le ocurra el caso correcto, sino de que dos implementaciones independientes coincidan.

`aceptar_fuente.py` Tokeniza un fuente real con el analizador léxico de la Etapa 1 y le pasa la secuencia al simulador, para poder probar la gramática contra los casos de la cátedra.

`generar_corpus_fuzz.py` y `lote_sintactico.py` Generan el corpus: Grammarinator deriva y muta programas a partir de la gramática ANTLR, el oráculo JavaCC dicta el veredicto, y los casos escritos a mano se emiten con su expectativa declarada.